

OmniVL-Guard Pro: A Tool-Augmented Agent for Omnibus Vision-Language Forensics

Jinjie Shen^{*123} Zheng Huang^{*2} Yuchen Zhang⁴ Yujiao Wu⁵
Yaxiong Wang^{†13} Lechao Cheng¹³ Shengeng Tang¹ Tianrui Hui¹³ Nan Pu¹³ Zhun Zhong^{†13}

¹School of Computer Science and Information Engineering, Hefei University of Technology, Hefei, China

²Wuhan University, Wuhan, China ³Lab for Intelligence and vision (LION)

⁴Xi'an Jiaotong University ⁵CSIRO

^{*}Equal contribution. [†]Corresponding author: wangyx@hfut.edu.cn; zhunzhong007@gmail.com



Figure 1. This work tackles omnibus vision-language forgery detection, grounding, and segmentation (left). In this unified setting, models relying on parametric memory alone cannot handle manipulations that require external clues or fine-grained inspection. In response, we propose OmniVL-Guard Pro, a tool-augmented agent trained with Checker-Guided Agentic RL (CGARL), achieving evidence-grounded reasoning and consistent gains over SFT across all tasks (right).

Abstract

Existing vision-language forgery detection and grounding methods operate under a closed-world paradigm, assuming verification can be completed by the model alone. However, self-contained MLLMs are constrained by finite parametric knowledge, static training corpora, and limited perceptual resolution, creating a practical ceiling in dynamic open-world forensics—particularly for real-time event verification requiring external clues and forgery segmentation demanding fine-grained scrutiny of local manipulations. To address these limitations, we shift from scaling up the self-contained model toward reaching beyond it. We propose **OmniVL-Guard Pro**, a tool-augmented agent that extends unified forensics from closed-world prediction to open-world clues-driven reasoning. OmniVL-Guard Pro integrates a tool environment spanning real-time event search, local cropping and zooming, edge-anomaly screening, face detection, video frame extraction, and SAM3-based segmentation. To generate high-quality tool-reasoning trajectories, we introduce **Tree-Structured Self-Evolving Tool Trajectory Generation**, which produces diverse trajectories through seed guidance, guider-free self-evolution, and weakly-hinted hard sample synthesis, yielding the Full-Spectrum Tool Reasoning (FSTR) dataset for training. We further propose **Checker-Guided Agentic Reinforcement Learning (CGARL)**, which provides process-level supervision to penalize cases where the answer is correct but the reasoning is distorted. Extensive experiments demonstrate that OmniVL-Guard Pro achieves state-of-the-art performance across various tasks, and exhibits strong zero-shot generalization. The FSTR dataset and code for OmniVL-Guard Pro will be publicly released at <https://github.com/shen8424/OmniVL-Guard-Pro>.

1. Introduction

The rapid evolution of generative AI has significantly lowered the barrier for creating manipulated content across diverse modalities to infiltrate real-world social media ecosystems at an unprecedented pace (Haydarov et al., 2024; Li et al., 2024b; Abdelnabi et al., 2022). Consequently, forgery detection, manipulation localization, and forgery segmentation across primary modalities including images, videos, and text have garnered substantial attention (Shen et al., 2025; Liu et al., 2024). Despite recent advancements such as OmniVL-Guard (Shen et al., 2026), which establishes a unified framework for omnibus vision-language forgery detection and grounding, existing methods still mainly operate under a *closed-world* paradigm, which implicitly assumes that *the verification process can be completed by the model itself*.

However, self-contained MLLMs (DeepSeek-AI, 2025; Zhang et al., 2025) remain constrained by finite parameters, static training corpora, and limited native perceptual resolution, which impose a practical ceiling in dynamic open-world forensics. This limitation is evident in two representative settings: Real-time Event Verification requires up-to-date external clues beyond parametric memory, while Forgery Segmentation demands fine-grained scrutiny of local boundaries, texture discontinuities, facial details, and semantic inconsistencies for pixel-level localization. Although scaling model size or training data could partially enhance these abilities, such a “scaling-up” strategy is costly and insufficient for continuously evolving real-world forgeries. We therefore shift from merely scaling up the self-contained model to reaching out beyond it, enabling the model to retrieve real-world knowledge and invoke specialized perception tools when necessary. In this sense, these two tasks jointly motivate an open-world forensic agent that *dynamically acquires external knowledge and fine-grained visual artifacts to transcend the intrinsic limits of the base model*.

To this end, we first construct a tool-enhanced environment that covering key capabilities such as real-time event search, local cropping, and segmentation (Sec. 2). This environment enables the model to evolve from a self-contained predictor into an clues-seeking agent that actively gathers, examines, and integrates external clues.

With the tool environment as the base, the agent needs to decide when and which tools is invoked. Cultivating this ability requires high-quality tool-based reasoning trajectory data. Unlike pure-text Chain-of-Thought (CoT), tool-based reasoning trajectory simultaneously involves tool selection, parameter specification, observation understanding, and stopping decisions, making it substantially more difficult to construct. Even state-of-the-art MLLMs (Zhao et al., 2024; Team, 2025; Guo et al., 2025) still exhibit lim-

ited exploration quality in real tool environments, and thus cannot directly produce sufficiently high-quality training trajectories. Conversely, directly injecting GT or answer-aware cues would cause the model to organize tool usage around the known answer, thereby introducing hindsight bias (Shen et al., 2026). To address this challenge, we propose a **Tree-Structured Self-Evolving Forensic Tool Trajectory Generation** framework. In the seed stage, it efficiently constructs high-fidelity trajectories through tree expansion and branch pruning. It then gradually scales up the data through unguided self-evolution, while further strengthening long-tail hard samples with weak hints, thereby providing cold-start data for subsequent tool-augmented RL.

Taking the trajectory data, we finally train the model for detection, grounding and segmentation with the aid of external tools. However, merely teaching the model to invoke tools is still insufficient. The core of forgery detection, grounding, and segmentation lies not only in obtaining external clues/artifacts, but also in forming correct and consistent judgments based on such evidence. The model still exhibits a pseudo-success pattern where the *“answer is accidentally correct but the process is distorted”*: although the final answer happens to be correct, the intermediate tool calls, observational evidence, and final judgment do not form a consistent and verifiable support relation, and such deviations are difficult for the model itself to identify. To tackle this problem, we further propose **Checker-Guided Agentic Reinforcement Learning** (CGARL). Specifically, we introduce a *Checker* to build a Multi-Agent system, which provides process-level supervision over intermediate reasoning, tool usage, and observational evidence, explicitly assessing whether they jointly support the final judgment. Through this mechanism, the model is encouraged to genuinely organize its reasoning around evidence, rather than remaining at the stage of merely knowing how to call tools without properly using the acquired evidence.

Built upon the above designs, we propose **OmniVL-Guard Pro**, an open-world tool-augmented Agent model. By integrating Tree-Structured Self-Evolving Forensic Tool Trajectory Generation with Checker-guided process-supervised RL, OmniVL-Guard Pro advances OmniVL-Guard from a closed-world unified forensic model into an open-world evidence-driven intelligent agent. Overall, our main contributions are summarized as follows:

(1) **Open world Agent for Vision-Language Forensics.** We propose OmniVL-Guard Pro, a tool-augmented Agent that performs forgery detection, grounding, and segmentation with external support across text, image, and video modalities, supporting real-time event verification and fine-grained forgery segmentation. We also construct a benchmark for real-time event verification.

(2) **FSTR Dataset.** We construct *Full-Spectrum Tool Rea-*

soning dataset, consisting of FSTR_{sft} for cold-start SFT and FSTR_{rl} for subsequent tool-augmented RL. FSTR_{sft} contains high-fidelity tool-search trajectories, while FSTR_{rl} supports subsequent tool-augmented RL.

(3) **Checker-Guided Agentic RL.** We propose Checker-Guided Agentic RL, which introduces process-level supervision beyond answer-level rewards to encourage evidence-consistent reasoning. It mitigates the pseudo-success pattern where the “answer is correct but the process is distorted,” improving both Agent performance and reasoning-evidence consistency.

2. Tool-Enhanced Environment

To enable open-world evidence acquisition beyond intrinsic model capabilities, we construct a tool environment for unified vision-language forgery detection, grounding, and segmentation.

Real-time event search tool. This tool searches the web given a keyword query (e.g., time, person, location), returning the most relevant results and supporting clues for real-time fact verification.

Local cropping tool. Given normalized coordinates, this tool extracts the specified region as a new image context for further inspection.

Local zooming tool. Given a candidate region, this tool returns an enlarged view for fine-grained inspection of boundary discontinuities, texture anomalies, local blur, and subtle artifacts.

Edge-anomaly screening tool scans the image using local edge density, Laplacian energy, and neighborhood discrepancy, and returns candidate regions for cropping and zooming.

Face detection tool returns face bounding boxes based on InsightFace (Deng et al., 2021), supporting facial region inspection in portrait and person-centric media.

Video clip frame extraction tool. Given a candidate temporal segment, this tool extracts frames for further examination using the image-level tools above.

SAM3 segmentation tool runs SAM3 (Carion et al., 2025) to produce a pixel-level mask from model-derived prompts: bounding box, positive points inside the target, and negative points in the background.

3. Tree-Structured Self-Evolving Tool Trajectory Generation

High-quality tool-search trajectories are a critical prerequisite for the cold-start phase of tool-augmented reinforcement learning. However, constructing such data in unified vision-language scenarios faces a new **efficiency–bias dilemma**:

on the one hand, if we directly rely on general-purpose MLLMs to freely explore in real tool environments, they often produce a large number of low-quality trajectories since stable tool-use capabilities have not yet been established; on the other hand, if GT or answer-aware cues are injected to force-guide the search process, hindsight bias will be introduced, causing the model to learn answer-conditioned tool invocation patterns rather than transferable evidence-seeking capabilities. To address this issue, we propose a **Tree-Structured Self-Evolving Tool Trajectory Generation** strategy. Overall, this strategy consists of four stages, as shown in Figure 2.

3.1. Source Data Collection

We first collect raw data from a series of existing datasets to cover the major modalities and task types involved in unified vision-language forensics. Specifically, for the unimodal text domain, we adopt FakeNewsCorpus (Shu et al., 2020) and MCFEND (Li et al., 2024a); for the single-image modality, we select FakeClue (Wen et al., 2025), LOKI (Ye et al., 2025), ForgeryNet (He et al., 2021), OpenSDI (Wang et al., 2025), and AutoSplice (Jia et al., 2023); for the video modality, we use GenVideo (Chen et al., 2024), DVF (Song et al., 2024), and ForgeryNet (He et al., 2021); for the image–text modality, we adopt SAMM (Shen et al., 2025), MDSM (Zhang et al., 2025), DGM⁴ (Shao et al., 2023), and NewsCLippings (Luo et al., 2021); and for the text–video modality, we select FakeSV (Qi et al., 2023). These datasets jointly cover diverse tasks related to forgery detection, manipulation grounding, and semantic segmentation. For the real-time event verification task, we use RealFact constructed in the Appendix B. Finally, we partition all source data into three parts: $\mathcal{D}_s$ for SFT cold-start data construction, $\mathcal{D}_r$ for subsequent RL data construction, and $\mathcal{D}_t$ for in-domain testing. The selection of out-of-domain zero-shot test data is detailed in Sec. 5.

3.2. Tool Trajectory Seed Guidance

On a small-scale subset of $\mathcal{D}_s$, we first construct a high-fidelity seed set of tool-search trajectories. To this end, we define an MLLM expert pool

$$\mathcal{M} = \{\text{GPT5.4, Gemini3.1, Seed2.0}\}. \quad (1)$$

For each sample, we randomly select two distinct models from $\mathcal{M}$ to serve as the *Explorer* and the *Guider*, respectively. The *Explorer* is responsible for executing search in the real tool environment, while the *Guider*, with the final answer a^* injected, determines the validity of the tool attempt performed at the current step. Based on this design, seed trajectory generation is not a linear rollout along a single path, but rather a tree-structured search process composed of branch expansion and branch pruning, as shown in Figure 2(c).

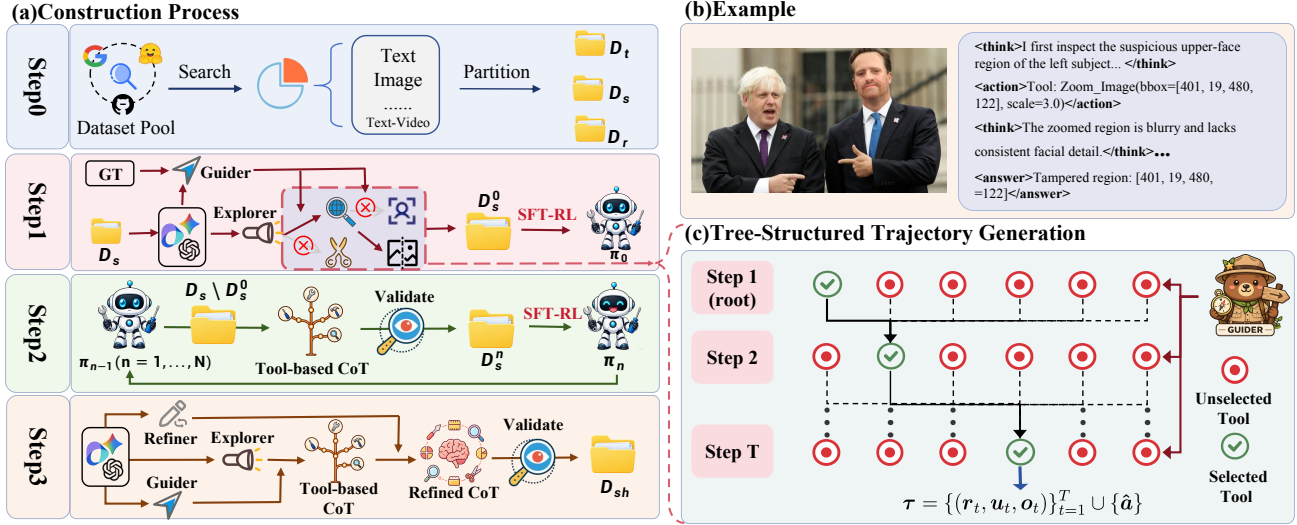


Figure 2. (a)The construction of FSTR_{sft}. (b) An example from FSTR_{sft}. (c) The process of Tree-Structured Trajectory Generation.

Formally, a tool-search trajectory is denoted as

$$\tau = \{(r_t, u_t, o_t)\}_{t=1}^T \cup \{\hat{a}\}, \quad (2)$$

where r_t denotes the model’s reasoning process at step t , which is formed based on previous tool-returned observations and determines the next tool to invoke as well as its parameters. u_t denotes the selected tool and its parameters at the current step, o_t denotes the observation returned by the tool, and $\hat{a}$ denotes the final prediction produced by the model at the end of the trajectory.

At step t , given the current history $\{(r_i, u_i, o_i)\}_{i=1}^{t-1}$, the *Explorer* expands the current node by sequentially attempting each tool in the tool set $\mathcal{T}$ (Sec. 2). For each candidate tool, the *Explorer* first generates the reasoning process r_t , which analyzes the previous observations and specifies both the tool to be invoked and its parameters, such as a search query, a crop box, or a zooming region. The corresponding tool invocation is denoted as u_t , and is executed in the real tool environment to obtain the returned observation o_t . The *Guider*, with the final answer a^* injected, then evaluates whether this candidate step (r_t, u_t, o_t) is helpful for solving the final task. If it is judged invalid, the current branch is immediately terminated, and the *Explorer* turns to the next candidate tool under the same history state; if it is judged valid, (r_t, u_t, o_t) is appended to the trajectory, and the resulting new state is further expanded in the next step following the same procedure.

After the *Explorer* completes the search and outputs the final prediction $\hat{a}$, we further conduct quality checking on the trajectory, requiring the final answer to be correct, the overall trajectory to be coherent, the intermediate reasoning and tool usage to support the final conclusion, and the trajectory to contain no obvious ineffective steps. The detailed criteria for the *Guider* judgment and the subsequent quality checking process are provided in Appendix C.1 C.2. All trajectories

that pass the checking process are included in the seed set $\mathcal{D}_s^0$. Subsequently, we perform SFT on Qwen3VL-8B (Yang et al., 2025) using $\mathcal{D}_s^0$, followed by RL on $\bar{\mathcal{D}}_r \subset \mathcal{D}_r$, yielding the initial policy π_0 .

3.3. Guider-free Self-Evolving Expansion

After obtaining the initial policy π_0 , we further design a Guider-free self-evolving expansion mechanism to continuously scale up high-quality tool-augmented reasoning trajectories. Specifically, in the n -th iteration, we use π_{n-1} to perform multi-turn rollout reasoning on $\mathcal{D}_s \setminus \mathcal{D}_s^0$, producing candidate tool trajectories. Different from the seed stage, we remove the answer-aware Guider from the rollout process, so the current policy autonomously explores the tool environment without Guider-based step-level judgment or branch pruning. For each candidate trajectory, we apply the same checking and filtering procedure as in Sec. 3.2, retaining only samples whose final answers are correct, whose trajectories are globally self-consistent, whose tool usage and reasoning processes support the final conclusion, and which contain no obvious ineffective steps (Appendix C.2). All trajectories that pass the filtering process are combined with $\mathcal{D}_s^0$ to form the current-round dataset $\mathcal{D}_s^n$. We then re-initialize SFT from Qwen3VL-8B using $\mathcal{D}_s^n$, followed by RL on $\bar{\mathcal{D}}_r$, obtaining the n -th round policy π_n . We repeat this self-evolving process for $N = 4$.

3.4. Weakly-Hinted Hard Trajectory Synthesis

Although self-evolution can substantially improve the coverage of valid trajectories, standard rollout could still fail to produce any accepted trajectory for samples whose correct tool-use paths require sparse clues, precise local regions, or non-obvious search keywords. These persistently failed samples constitute the long-tail portion of the trajectory dis-

tribution. To complement the self-evolved data on such hard cases, we further design a weakly-hinted hard trajectory synthesis mechanism.

Specifically, for samples that consistently fail during the self-evolution stage, following Sec. 3.2, we randomly select two models from $\mathcal{M}$ to serve as *Explorer* and *Guider*, respectively. The *Explorer* sequentially attempts each tool in the tool set $\mathcal{T}$ at each step to generate candidate trajectories, while the *Guider*, with the final answer a^* injected, judges the tool attempt at the current step. Different from Sec. 3.2, the *Guider* not only determines whether the current tool selection is valid, but also provides usage hints for the tool, namely necessary parameters or local clues related to the current tool invocation, to narrow down the search space at the current step for the *Explorer*. For example, in image scenarios, the *Guider* provides a rough candidate region to assist cropping or zooming. Based on these weak hints, the *Explorer* performs tool search and generates candidate trajectories.

After obtaining the weakly-hinted candidate trajectories, we further randomly select another model from $\mathcal{M}$ to serve as the *Refiner*, which rewrites the reasoning process r_t at each step of the trajectory to eliminate explicit traces introduced by weak hints, preventing such answer-aware hints from contaminating the trajectory and inducing hint-dependent reasoning. We then apply the same quality checking procedure as in Sec. 3.2. All trajectories that pass the checking process are included in the hard-sample set $\mathcal{D}_{sh}$. The rewriting details are provided in Appendix C.4.

3.5. Dataset Statistics

Through the aforementioned process, we obtain the Full-Spectrum Tool Reasoning (FSTR) dataset. It consists of two parts: the tool-search trajectory set for cold-start SFT, $\text{FSTR}_{\text{sft}} = \mathcal{D}_s^N \cup \mathcal{D}_{sh}$; and the dataset for RL training, $\text{FSTR}_{\text{rl}} = \mathcal{D}_r \setminus \bar{\mathcal{D}}_r$. FSTR_{sft} contains 69,323 samples, where $\mathcal{D}_{sh}$ accounts for approximately 17% and $\mathcal{D}_s^0$ accounts for approximately 8%; FSTR_{rl} contains 81,087 samples. FSTR covers five core modalities, including text, image, video, text-image, and text-video, and supports full-spectrum vision-language classification tasks, omni-modal grounding tasks, image-modal segmentation tasks, and real-time event verification tasks.

4. Checker-Guided Agentic Reinforcement Learning

After obtaining the cold-start tool trajectory data, we adopt a three-stage training strategy. Specifically, we first perform SFT on FSTR_{sft} and then split FSTR_{rl} into two disjoint subsets, $\mathcal{D}_{\text{rl}}^1$ and $\mathcal{D}_{\text{rl}}^2$. In the first stage, we conduct outcome-level RL on $\mathcal{D}_{\text{rl}}^1$, yielding a tool-capable policy π_{ans} . In the

second stage, we rollout π_{ans} on $\mathcal{D}_{\text{rl}}^2$ to construct process supervision data and train the process discrimination model π_{checker} . The third stage further performs Checker-guided process-supervised RL on $\mathcal{D}_{\text{rl}}^2$, jointly improving answer correctness, process consistency, and tool-use efficiency. The rationale behind this design is that effective Checker supervision requires a policy with stable tool-use capabilities; otherwise, process-level signals would be noisy when the model still fails to perform reliable detection, grounding, or segmentation.

4.1. Stage I: Outcome-Driven RL with ARSPO

We first perform SFT on Qwen3VL-8B using FSTR_{sft} , obtaining the initial policy π_{sft} . For a sample with a tool trajectory, we write it as

$$\tau = \{(r_t, u_t, o_t)\}_{t=1}^T \cup \{\hat{a}\}, \quad (3)$$

where the definitions of r_t , u_t , o_t , and $\hat{a}$ are consistent with Eq. 2. Since o_t is returned by the environment rather than generated by the model, we do not compute loss over these tokens during SFT. Instead, the language modeling loss is only applied to model-generated tokens corresponding to r_t , u_t and $\hat{a}$.

Let the complete token sequence of τ , including all T reasoning-tool interaction steps and the final answer, be $y = (y_1, \dots, y_{|y|})$, and define a token-level mask $m_i \in \{0, 1\}$, where positions corresponding to tool-returned observations o_t take $m_i = 0$, and all other model-generated positions take $m_i = 1$. The SFT loss is therefore defined as

$$\mathcal{L}_{\text{sft}} = - \sum_{(x, \tau) \in \text{FSTR}_{\text{sft}}} \frac{1}{\sum_{i=1}^{|y|} m_i} \times \sum_{i=1}^{|y|} m_i \log \pi_{\theta}(y_i \mid x, y_{<i}). \quad (4)$$

In this way, the model learns how to generate reasonable tool invocations, parameter specifications, intermediate reasoning, and final answers, rather than fitting the contents returned by the tools.

Subsequently, we split FSTR_{rl} into two parts, $\mathcal{D}_{\text{rl}}^1$ and $\mathcal{D}_{\text{rl}}^2$, where $\mathcal{D}_{\text{rl}}^1$ is used for outcome-level RL, and $\mathcal{D}_{\text{rl}}^2$ is reserved for subsequent Checker-guided process-supervised optimization.

For each task question x in $\mathcal{D}_{\text{rl}}^1$, the current policy π_{θ} performs inference in the real tool environment and produces a complete response $y = (\tau, \hat{a})$. Then we define the total reward as

$$R(y, x) = R_{\text{ans}}(y, x) + R_{\text{fmt}}(y) + R_{\text{rep}}(y), \quad (5)$$

where R_{ans} , R_{fmt} , and R_{rep} denote the answer reward, format reward, and repetition penalty, respectively. R_{ans}

is computed by accuracy for classification tasks, and by overlap-based metrics such as IoU or Dice for grounding and segmentation tasks. Details are provided in Appendix D.1.

We adopt ARSPO (Shen et al., 2026) to update the policy, whose optimization objective is formulated as follows:

$$\mathcal{J}_{\text{arspo}}(\theta) = \sum_{k=1}^K \frac{|\mathcal{D}_{\text{rl},k}^1|}{|\mathcal{D}_{\text{rl}}^1|} \mathbb{E}_{\substack{x \sim \mathcal{D}_{\text{rl},k}^1, \\ \{y_i\} \sim \pi_{\theta_{\text{old}}}}} \left[\frac{l_{k,s}}{G} \sum_{i=1}^G \frac{1}{|y_i|} \sum_{t=1}^{|y_i|} f_{i,t}(r_{i,t}(\theta)) \hat{A}_{i,k} \right], \quad (6)$$

where K denotes the number of task categories, $\mathcal{D}_{\text{rl},k}^1 \subset \mathcal{D}_{\text{rl}}^1$ denotes the data subset corresponding to the k -th task, and G denotes the number of sampled responses for each question. y_i denotes the i -th complete response, and $|y_i|$ is its total token length. $\hat{A}_{i,k} = \frac{A_{i,k} - \mu}{\sigma}$ is the normalized advantage, with μ and σ being the mean and standard deviation of the rewards $\{A_{i,k}\}_{i=1}^G$. $f_{i,t}(\cdot)$ is the token-level weighting function in the form of SAPO (Gao et al., 2025) (Detailed in Appendix D.2), and $l_{k,s}$ denotes the dynamic coefficient for the k -th task at training step s . $r_{i,t} = \frac{\pi_{\theta}(y_{i,t}|q, y_{i,<t})}{\pi_{\theta_{\text{old}}}(y_{i,t}|q, y_{i,<t})}$ is the probability ratio.

Consistent with the SFT stage, the token-level optimization in Eq. (6) is only applied to model-generated tokens. After this stage, the model is updated from π_{sft} to π_{ans} , which serves as the initialization policy for subsequent Checker-guided process-supervised optimization.

4.2. Stage II: Checker Construction via Rollout Distillation

After obtaining π_{ans} , we construct a Checker to evaluate whether the complete tool reasoning trajectory produced by the Detector provides sufficient and consistent support for its final prediction, thereby avoiding the pseudo-success pattern where the ‘‘answer is correct but the process is distorted.’’ For each sample, the Checker takes the question x , the trajectory τ , and the predicted answer $\hat{a}$ as input, and outputs fine-grained evaluation together with an overall score s .

We define the scoring space as three discrete levels: $s \in \{0, 0.5, 1\}$. Specifically, $s = 1$ indicates that the entire trajectory is internally consistent, the tool usage is reasonable, and the acquired key evidence is sufficient to support the final prediction; $s = 0.5$ indicates that the trajectory is locally reasonable but still suffers from issues such as insufficient evidence or incomplete intermediate support; and $s = 0$ indicates that the trajectory lacks effective support overall, the tool usage and intermediate reasoning fail to form a self-consistent evidence chain, or the final prediction is clearly inconsistent with the reasoning process. Details are provided in Appendix F.1.

We adopt a discrete three-level score rather than a continuous dense score because the Checker is intended to provide process supervision signals with clear boundaries and stable judgments. In contrast, continuous scores often lack fine-grained discriminative criteria; for instance, the model cannot reliably provide a distinguishable boundary between 0.95 and 0.93, which would introduce additional noise.

For each question x in $\mathcal{D}_{\text{rl}}^2$, we use π_{ans} to perform rollout in the real tool environment, obtaining a complete response $y = (\tau, \hat{a})$. Based on these rollout responses, we further construct supervised samples for the Checker.

First, we perform preliminary filtering on the raw responses produced by π_{ans} rollout, removing samples with abnormal formats, incomplete trajectories, or parsing failures. We then construct an MLLM pool $\mathcal{M} = \{\text{GPT5.4, Gemini3.1, Seed2.0}\}$, and randomly select two MLLMs from it. Each selected MLLM is required to assess the trajectory according to the scoring criteria defined above, outputting both the fine-grained evaluation rationale and the discrete score s . Samples for which the two scores disagree are removed to complete cross-verification. Next, the remaining MLLM verifies the above evaluations and scores, filtering out samples with ambiguous evidence or unclear scoring boundaries. All samples that pass the filtering process jointly constitute the Checker training set $\mathcal{D}_{\text{checker}}$. The construction details are provided in Appendix F.2.

We then concatenate the fine-grained evaluation and the discrete score into the target output sequence, and perform standard autoregressive SFT on Qwen3VL-8B using $\mathcal{D}_{\text{checker}}$, obtaining the process discrimination model π_{checker} .

4.3. Stage III: Checker-Guided Process RL with ARSPO

After obtaining the outcome-level policy π_{ans} and the process discrimination model π_{checker} , we further conduct Checker-guided process-supervised RL, encouraging the model to generate more coherent, trustworthy, and concise tool reasoning trajectories while maintaining answer correctness.

For each task question x in $\mathcal{D}_{\text{rl}}^2$, the current policy performs inference in the real tool environment and produces $y = (\tau, \hat{a})$. We first determine whether the response is correct based on the final answer. For classification tasks, correctness means that the final prediction matches the annotation; for grounding and segmentation tasks, correctness means that the corresponding evaluation metric is higher than 0.7. Only correct responses are further evaluated by π_{checker} to obtain a process score $s_{\text{proc}} \in \{0, 0.5, 1\}$, while incorrect responses are not subjected to Checker-based process penalization, since they have already been penalized by the answer reward.

To avoid unconstrained tool invocation during inference,

we further design a tool-use efficiency reward. Specifically, among all correct responses to the same question, the response with the fewest tool calls receives $R_{\text{toolmin}}(y) = 0.5$, while the others receive $R_{\text{toolmin}}(y) = 0$.

Based on the above design, we define the total reward as

$$R(y, x) = \begin{cases} s_{\text{proc}}(y) \times (R_{\text{ans}}(y, x) + R_{\text{fmt}}(y) \\ \quad + R_{\text{toolmin}}(y) + R_{\text{rep}}(y)), & \text{if correct,} \\ R_{\text{ans}}(y, x) + R_{\text{rep}}(y) + R_{\text{fmt}}(y), & \text{otherwise.} \end{cases} \quad (7)$$

where R_{ans} , R_{toolmin} , R_{rep} , and R_{fmt} denote the answer reward, tool-use efficiency reward, repetition penalty, and format reward, respectively.

We still adopt ARSPO to update the policy, with the optimization objective defined as follows:

$$\begin{aligned} \mathcal{J}_{\text{arspo}}^{\text{proc}}(\theta) = & \sum_{k=1}^K \frac{|\mathcal{D}_{\text{rl},k}^2|}{|\mathcal{D}_{\text{rl}}^2|} \mathbb{E}_{\substack{x \sim \mathcal{D}_{\text{rl},k}^2, \\ \{y_i\} \sim \pi_{\theta_{\text{old}}}}} \\ & \times \left[\frac{l_{k,s}}{G} \sum_{i=1}^G \frac{1}{|y_i|} \sum_{t=1}^{|y_i|} f_{i,t}(r_{i,t}(\theta)) \hat{A}_{i,k} \right], \end{aligned} \quad (8)$$

The token-level optimization in Eq. (8) is only applied to model-generated tokens. After this stage, the model is jointly optimized in terms of answer correctness, process consistency, and tool-use efficiency, ultimately yielding the process-supervision-enhanced policy.

During RL rollout and evaluation, each generated tool call is executed online, and its observation is fed back before the next step; see Appendix E.

5. Experiments

Implementation details are provided in Appendix D.

Benchmarks and Baselines. We evaluate on both In-Domain and Out-of-Domain (OOD) benchmarks. The In-Domain setting uses the test split from Sec. 3.1, while the OOD setting is designed to assess zero-shot generalization. The OOD suite includes ISOT (Ahmed et al., 2017) for text, CASIA2.0 (Yadav & Vishwakarma, 2024) for image, MM-FakeBench (Liu et al., 2025) for image-text, FakeTT (Bu et al., 2024) for video-text, and IMD (Chen et al., 2025) for segmentation. For baselines, general MLLMs (e.g., Qwen3VL-235B) are evaluated in a zero-shot manner. In contrast, domain-specific methods are trained and then directly evaluated on OOD benchmarks without further fine-tuning, in order to assess cross-domain robustness.

5.1. Main Results

In-Domain Comparison. As shown in Table ??, OmniVL-Guard Pro achieves the best in-domain performance across detection, localization, real-time verification, and segmentation. While maintaining strong binary classification across all modalities, it delivers larger gains on fine-grained tasks, improving RealFact verification and segmentation by 30.71% and 25.77% over the prior results, validating the benefits of tool-augmented evidence acquisition and Checker-guided process supervision.

Out-of-Domain Zero-Shot Comparison. As shown in Table ??, OmniVL-Guard Pro achieves the strongest OOD zero-shot performance across unseen benchmarks, outperforming both general MLLMs and domain-specific models. It attains 81.74% and 68.06% accuracy on Text-Image and Text-Video tasks, respectively, and reaches 42.93% Dice on IMD segmentation. These results demonstrate that self-evolving tool reasoning and CGARL improve robustness to unseen multimodal forgeries and generalize beyond classification to fine-grained segmentation.

5.2. Ablation Study

Ablation of Training Components. As shown in Table ??, SFT alone provides only limited tool-use capability, while adding ARSPO brings a strong average gain of 21.64%, confirming the importance of outcome-level RL. Directly using Qwen3VL-8B as the intermediate process supervisor without training a dedicated Checker is less effective, yielding only 16.28% average improvement, which suggests that generic MLLM feedback can introduce noisy process signals. With the trained Checker and Checker-guided RL, the full model achieves the best performance across all tasks and the largest average gain of 28.16%, validating the necessity of task-adapted process supervision.

Ablation of Tool Modules. Figure ?? shows that the full tool set consistently achieves the best overall performance, while removing all tools causes large drops across all tasks, confirming the necessity of external tool interaction. In particular, search is crucial for RealFact verification, where removing it reduces accuracy from 86.5% to 62.3%. For video localization, removing the video tool leads to the largest degradation, reducing tIoU from 64.4% to 15.7%. These results indicate that different tools provide complementary evidence for different forensic subtasks.

More Discussion. We further investigate the impact of the self-evolution iteration rounds N on model performance (Appendix G) and the necessity of the Weakly-Hinted Hard Trajectory Synthesis for covering long-tail hard samples (Appendix H).

6. Conclusion

We present OmniVL-Guard Pro, a tool-augmented agent that extends unified vision-language forensics from closed-world prediction to open-world evidence-driven reasoning. Through Tree-Structured Self-Evolving Trajectory Generation and Checker-Guided Agentic Reinforcement Learning, our model learns to actively acquire external clues, perform fine-grained inspection, and form evidence-consistent judgments. OmniVL-Guard Pro achieves state-of-the-art performance across all tasks, with strong zero-shot generalization.

References

- Abdelnabi, S., Hasan, R., and Fritz, M. Open-domain, content-based, multi-modal fact-checking of out-of-context images via online resources. In *CVPR*, pp. 14920–14929, 2022.
- Ahmed, H., Traoré, I., and Saad, S. Detection of online fake news using n-gram analysis and machine learning techniques. In *ISDDC*, pp. 127–138, 2017.
- Bu, Y., Sheng, Q., Cao, J., Qi, P., Wang, D., and Li, J. Fakingrecipe: Detecting fake news on short video platforms from the perspective of creative process. In *ACM MM*, pp. 1351–1360, 2024.
- Carion, N., Gustafson, L., Hu, Y., Debnath, S., Hu, R., Suris, D., Ryali, C., Alwala, K. V., Khedr, H., Huang, A., Lei, J., Ma, T., Guo, B., Kalla, A., Marks, M., Greer, J., Wang, M., Sun, P., Rädle, R., Afouras, T., Mavroudi, E., Xu, K., Wu, T., Zhou, Y., Momeni, L., Hazra, R., Ding, S., Vaze, S., Porcher, F., Li, F., Li, S., Kamath, A., Cheng, H. K., Dollár, P., Ravi, N., Saenko, K., Zhang, P., and Feichtenhofer, C. SAM 3: Segment anything with concepts. *CoRR*, 2025.
- Chen, H., Hong, Y., Huang, Z., Xu, Z., Gu, Z., Li, Y., Lan, J., Zhu, H., Zhang, J., Wang, W., and Li, H. Demamba: Ai-generated video detection on million-scale genvideo benchmark. *CoRR*, 2024.
- Chen, Y., Chang, M., Kirchner, M., Zhang, Z., Li, X., Basharat, A., and Hoogs, A. A semantically impactful image manipulation dataset: Characterizing image manipulations using semantic significance. In *WACV*, pp. 7659–7668, 2025.
- DeepSeek-AI. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *CoRR*, 2025.
- Deng, J., Guo, J., An, X., Zhu, Z., and Zafeiriou, S. Masked face recognition challenge: The insightface track report. In *ICCV Vision Workshops*, pp. 1437–1444, 2021.
- Gao, C., Zheng, C., Chen, X., Dang, K., Liu, S., Yu, B., Yang, A., Bai, S., Zhou, J., and Lin, J. Soft adaptive policy optimization. *CoRR*, 2025.
- Guo, D., Wu, F., Zhu, F., Leng, F., Shi, G., Chen, H., Fan, H., Wang, J., Jiang, J., Wang, J., Chen, J., Huang, J., Lei, K., Yuan, L., Luo, L., Liu, P., Ye, Q., Qian, R., Yan, S., Zhao, S., Peng, S., Li, S., Yuan, S., Wu, S., Cheng, T., Liu, W., Wang, W., Zeng, X., Liu, X., Qin, X., Ding, X., Xiao, X., Zhang, X., Zhang, X., Xiong, X., Peng, Y., Chen, Y., Li, Y., Hu, Y., Lin, Y., Hu, Y., Zhang, Y., Wu, Y., Li, Y., Liu, Y., Ling, Y., Qin, Y., Wang, Z., He, Z., Zhang, A., Yi, B., Liao, B., Huang, C., Zhang, C., Deng, C., Deng, C., Lin, C., Yuan, C., Li, C., Gou, C., Lou, C., Wei, C., Liu, C., Li, C., Zhu, D., Zhong, D., Li, F., Zhang, F., Wu, G., Li, G., Xiao, G., Lin, H., Yang, H., Wang, H., Ji, H., Hao, H., Shen, H., Li, H., Li, J., Wu, J., Zhu, J., Jiao, J., Feng, J., Chen, J., Duan, J., Liu, J., Zeng, J., Tang, J., Sun, J., Chen, J., Long, J., Feng, J., Zhan, J., Fang, J., Lu, J., Hua, K., Liu, K., Shen, K., Zhang, K., and Shen, K. Seed1.5-vl technical report. *CoRR*, 2025.
- Haydarov, K., Muhamed, A., Shen, X., Lazarevic, J., Skorokhodov, I., Galappaththige, C. J., and Elhoseiny, M. Adversarial text to continuous image generation. In *CVPR*, pp. 6316–6326, 2024.
- He, Y., Gan, B., Chen, S., Zhou, Y., Yin, G., Song, L., Sheng, L., Shao, J., and Liu, Z. Forgerynet: A versatile benchmark for comprehensive forgery analysis. In *CVPR*, pp. 4360–4369, 2021.
- Jia, S., Huang, M., Zhou, Z., Ju, Y., Cai, J., and Lyu, S. AutosplICE: A text-prompt manipulated image dataset for media forensics. In *CVPR*, pp. 893–903, 2023.
- Li, Y., He, H., Bai, J., and Wen, D. MCFEND: A multi-source benchmark dataset for chinese fake news detection. In *WWW*, pp. 4018–4027, 2024a.
- Li, Z., Tucker, R., Snaveley, N., and Holynski, A. Generative image dynamics. In *CVPR*, pp. 24142–24153, 2024b.
- Liu, X., Li, P., Huang, H., Li, Z., Cui, X., Liang, J., Qin, L., Deng, W., and He, Z. Fka-owl: Advancing multimodal fake news detection through knowledge-augmented lvlms. In *ACM MM*, pp. 10154–10163. ACM, 2024.
- Liu, X., Li, Z., Li, P., Huang, H., Xia, S., Cui, X., Huang, L., Deng, W., and He, Z. Mmfakebench: A mixed-source multimodal misinformation detection benchmark for lvlms. In *ICLR*, 2025.
- Luo, G., Darrell, T., and Rohrbach, A. Newsclippings: Automatic generation of out-of-context multimodal media. In *EMNLP*, pp. 6801–6817, 2021.
- Qi, P., Bu, Y., Cao, J., Ji, W., Shui, R., Xiao, J., Wang, D., and Chua, T. Fakesv: A multimodal benchmark with rich social context for fake news detection on short video platforms. In *AAAI*, pp. 14444–14452, 2023.
- Shao, R., Wu, T., and Liu, Z. Detecting and grounding multi-modal media manipulation. In *CVPR*, pp. 6904–6913. IEEE, 2023.
- Shen, J., Wang, Y., Cheng, L., Pu, N., and Zhong, Z. Beyond artificial misalignment: Detecting and grounding semantic-coordinated multimodal manipulations. In *ACM MM*, 2025.
- Shen, J., Wu, J., Wang, Y., Cheng, L., Tang, S., Hui, T., Pu, N., and Zhong, Z. Omnivl-guard: Towards unified vision-language forgery detection and grounding via balanced RL. *CoRR*, 2026.
- Shu, K., Mahudeswaran, D., Wang, S., Lee, D., and Liu, H. Fake-newsnet: A data repository with news content, social context, and spatiotemporal information for studying fake news on social media. *Big Data*, 8(3):171–188, 2020.
- Song, X., Guo, X., Zhang, J., Li, Q., Bai, L., Liu, X., Zhai, G., and Liu, X. On learning multi-modal forgery representation for diffusion generated video detection. In *NeurIPS*, 2024.

- Team, G. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *CoRR*, 2025.
- Wang, Y., Huang, Z., and Hong, X. Opensdi: Spotting diffusion-generated images in the open world. In *CVPR*, pp. 4291–4301, 2025.
- Wen, S., Ye, J., Feng, P., Kang, H., Wen, Z., Chen, Y., Wu, J., Wu, W., He, C., and Li, W. Spot the fake: Large multimodal model-based synthetic image detection with artifact explanation. In *NeurIPS*, 2025.
- Yadav, A. and Vishwakarma, D. K. Towards effective image forensics via A novel computationally efficient framework and A new image splice dataset. *CoRR*, 2024.
- Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., Zheng, C., Liu, D., Zhou, F., Huang, F., Hu, F., Ge, H., Wei, H., Lin, H., Tang, J., Yang, J., Tu, J., Zhang, J., Yang, J., Yang, J., Zhou, J., Lin, J., Dang, K., Bao, K., Yang, K., Yu, L., Deng, L., Li, M., Xue, M., Li, M., Zhang, P., Wang, P., Zhu, Q., Men, R., Gao, R., Liu, S., Luo, S., Li, T., Tang, T., Yin, W., Ren, X., Wang, X., Zhang, X., Ren, X., Fan, Y., Su, Y., Zhang, Y., Zhang, Y., Wan, Y., Liu, Y., Wang, Z., Cui, Z., Zhang, Z., Zhou, Z., and Qiu, Z. Qwen3 technical report. *CoRR*, abs/2505.09388, 2025.
- Ye, J., Zhou, B., Huang, Z., Zhang, J., Bai, T., Kang, H., He, J., Lin, H., Wang, Z., Wu, T., Wu, Z., Chen, Y., Lin, D., He, C., and Li, W. LOKI: A comprehensive synthetic data detection benchmark using large multimodal models. In *ICLR*, 2025.
- Zhang, Y., Wang, Y., Wu, Y., Wu, L., and Zhu, L. The coherence trap: When mllm-crafted narratives exploit manipulated visual contexts. In *CVPR*, 2025.
- Zhao, J., Wang, T., Abid, W., Angus, G., Garg, A., Kinnison, J., Sherstinsky, A., Molino, P., Addair, T., and Rishi, D. Lora land: 310 fine-tuned llms that rival gpt-4, A technical report. *CoRR*, 2024.

Appendix

A.	Related Work	11
B.	Real-time Event Benchmark	11
	B.1 Construction Process	11
	B.2 Prompt Templates	12
	B.3 Human Verification	14
	B.4 RealFact Benchmark Examples	14
C.	Details of Tool Trajectory Seed Guidance	15
	C.1 Guider Judgment in Seed Trajectory Construction	15
	C.2 Quality Checking Process	15
	C.3 Guider in Weakly-Hinted Hard Trajectory Synthesis	16
	C.4 Refiner for Hint Trace Elimination	17
D.	Implementation Details	18
	D.1 Reward Function Configuration	18
	D.2 Advantage Weighting Function	19
	D.3 Hyperparameter Settings in ARSPO	20
E.	Inference Protocol for Tool Interaction	20
F.	Details of Checker Construction	20
	F.1 Scoring Criteria for Process Quality	21
	F.2 MLLM Prompts for Checker Construction	22
G.	Analysis of Self-Evolution Iteration Rounds	23
H.	Ablation Study on Weakly-Hinted Hard Trajectory Synthesis	23
I.	Limitations	24
	I.1 Computational Cost and Open-Source Commitment	24
	I.2 Safeguards Against Misuse	24
J.	Case Study	24

A. Related Work

Forgery Detection and Grounding. With the rapid evolution of generative techniques, detection methodologies have progressed from unimodal paradigms—targeting specific modalities such as images, text, or videos—to bimodal synergies (e.g., image–text, video–text). More recently, OmniVL-Guard establishes a unified framework that simultaneously handles detection and grounding across image, video, and text modalities within a single model. Nevertheless, all these methods operate under a closed-world paradigm that implicitly assumes verification can be completed by the model itself, leaving them vulnerable to manipulations requiring external evidence or fine-grained perceptual scrutiny.

Tool-Augmented Language Agents. The integration of tool use into large language models has given rise to a new class of agents that interact with external environments to accomplish complex tasks. Foundational frameworks such as ReAct and Toolformer demonstrate that LLMs can effectively interleave reasoning with tool invocation. This paradigm has been extended to multimodal settings, where agents leverage specialized tools for visual question answering, image editing, and embodied reasoning. Despite these advances, the application of tool-augmented agents to forensics remains largely unexplored. Forgery detection, grounding, and segmentation present unique challenges: the agent must decide not only when and which tools to invoke, but also how to integrate heterogeneous evidence from diverse sources into a coherent and verifiable judgment.

OmniVL-Guard Pro. Our work extends OmniVL-Guard from a closed-world unified forensic model to an open-world evidence-driven agent. Unlike prior methods that rely solely on parametric knowledge, OmniVL-Guard Pro actively retrieves real-time evidence through web search, performs fine-grained local inspection via cropping and zooming tools, and produces pixel-level manipulation masks using SAM3. To train this agent, we propose Tree-Structured Self-Evolving Trajectory Generation, which addresses the efficiency–bias dilemma inherent in constructing tool-reasoning data: direct MLLM exploration yields low-quality trajectories, while injecting ground-truth cues introduces hindsight bias. Furthermore, we introduce Checker-Guided Agentic Reinforcement Learning (CGARL) to provide process-level supervision that penalizes the pseudo-success pattern where the answer is accidentally correct but the reasoning process is distorted, a failure mode that answer-level rewards alone cannot detect.

B. Real-time Event Benchmark

B.1. Construction Process

To evaluate whether models possess real-time event verification capabilities beyond parametric memory, we construct a real-time event benchmark **RealFact** for open-world fact verification. The benchmark covers the period from January 1, 2025 to March 31, 2026, containing 6,208 positive samples and 6,208 negative samples across diverse domains, including politics, economy, technology, society, sports, and more. The construction process is detailed as follows:

▷ *Data Collection:* We first collect candidate real-time events from publicly accessible information sources to build an initial event pool. The sources include mainstream news media, official announcements, as well as public webpages such as sports announcements and policy release pages. Each candidate event is required to satisfy three criteria simultaneously: first, the event time must be explicit and fall within the target time window; second, publicly accessible external textual evidence must be available and cross-supported by multiple independent sources; third, the event must contain verifiable key elements, such as persons, organizations, locations, time, actions, or outcomes. Based on these criteria, we clean and deduplicate the raw candidate events, removing samples with ambiguous timestamps, single-source evidence, or insufficient verifiable support. The retained events constitute the base event set of the benchmark, covering politics, economy, technology, society, sports, disasters, international news, and other domains.

▷ *Evidence-grounded Sample Construction:* After obtaining the base event set, we employ an MLLM to extract key information from each event, forming structured information that includes the event time, subject, location, key action, which is subsequently verified by another MLLM. Based on the structured information, we construct benchmark samples. For positive samples, the textual claim is consistent with the canonical event representation and can be directly supported by external evidence. For negative samples, we construct deceptive claims around real events, rather than using fabricated statements that are obviously incorrect or semantically incoherent. Specifically, these negative samples are created by modifying the event time, replacing the event subject, altering location information, changing the key action or outcome, or incorrectly stitching together multiple real events.

▷ *Annotation and Verification:* To ensure the quality and difficulty of the benchmark, we further conduct multi-stage filtering on the constructed samples. First, we use GPT-5.4 to check whether samples contain explicit cues that directly leak the answer, and remove samples with obvious hinting information. Next, we use Qwen3VL-235B, whose knowledge base is up to early 2025, to reason over all samples without providing external evidence, filtering out simple samples that can be judged solely from parametric memory. Then, we provide Qwen3VL-235B with the corresponding external evidence and perform inference again, further removing ambiguous samples for which a clear conclusion still cannot be obtained even with evidence. Only samples that pass all the above checks are included in the final benchmark. Finally, we conduct human verification on a randomly sampled subset to validate the quality of the automated construction and filtering pipeline.

B.2. Prompt Templates

This section provides the detailed prompt templates used in the benchmark construction process.

Prompt 1: Key Information Extraction

System Instruction: You are an expert information extraction assistant. Your task is to extract structured information from news articles about real-world events.

Your Task: Extract the following information from the provided news article:

1. *Event Time:* The specific date or time when the event occurred.
2. *Subject:* The person, organization, or entity involved in the event.
3. *Location:* The place where the event took place.
4. *Key Action:* The main action or event that occurred.
5. *Outcome:* The result or consequence of the event.

Input Data: News Article: {article.text}

Output Format: Return strictly in JSON format: { "time": "...", "subject": "...", "location": "...", "action": "...", "outcome": "..." }

Prompt 2: Information Verification

System Instruction: You are an expert fact-checking assistant. Your task is to verify whether extracted information accurately reflects the original news article.

Your Task: Verify each field of the extracted information against the original article:

1. Is the event time correctly extracted?
2. Is the subject correctly identified?
3. Is the location correctly identified?
4. Is the key action correctly described?
5. Is the outcome correctly stated?

Input Data: News Article: {article.text} **Extracted Information:** {extracted.json}

Output Format: For each field, respond with "Correct" or "Incorrect" and provide a brief explanation if incorrect.

Prompt 3: Leakage Check (GPT-5.4)

System Instruction: You are an expert quality assurance assistant. Your task is to check whether benchmark samples contain explicit cues that directly leak the answer.

Your Task: Analyze the claim and evidence to detect any leakage:

1. Does the claim contain obvious factual errors that can be detected without verification?
2. Does the claim contain self-contradictory information?
3. Does the claim contain explicit hints about its truthfulness?

Input Data: Claim: {claim_text} **Evidence:** {evidence_text}

Output Format: Respond with “No Leakage” if the claim does not contain obvious cues, or “Leakage Detected” with explanation.

Prompt 4: Parametric Memory Reasoning (Qwen3VL-235B)

System Instruction: You are a knowledgeable assistant. Based solely on your internal knowledge (without any external evidence), determine whether the given claim is true or false.

Your Task: Analyze the claim and provide your judgment:

1. Consider the factual accuracy of the claim.
2. Consider the temporal consistency (does the time match known events?).
3. Consider the plausibility of the described event.

Input Data: Claim: {claim_text}

Output Format: Respond with:

- “True” if you believe the claim is factually correct.
- “False” if you believe the claim is factually incorrect.
- “Uncertain” if you cannot determine the truthfulness from your knowledge.

Prompt 5: Evidence-Grounded Reasoning (Qwen3VL-235B)

System Instruction: You are an expert fact-verification assistant. Based on the provided external evidence, determine whether the given claim is true or false.

Your Task: Analyze the claim against the evidence:

1. Does the evidence support the claim?
2. Is there any contradiction between the claim and the evidence?
3. Is the evidence sufficient to make a definitive judgment?

Input Data: Claim: {claim_text} **Evidence:** {evidence_text}

Output Format: Respond with:

- “Supported” if the evidence confirms the claim is true.
- “Refuted” if the evidence contradicts the claim.
- “Insufficient” if the evidence is not enough to make a judgment.

B.3. Human Verification

To validate the quality of the automated construction and filtering pipeline, we conduct human verification on a randomly sampled subset of the benchmark. This subsection details the verification process, criteria, and results.

▷ *Verification Criteria:*

1. **Claim Authenticity:** Does the claim accurately reflect the original event? For positive samples, the claim should be factually consistent with the event. For negative samples, the introduced manipulation should be realistic and non-trivial.
2. **Evidence Sufficiency:** Is the provided evidence sufficient to verify or refute the claim? The evidence should be directly relevant and conclusive.
3. **Temporal Consistency:** Is the event timestamp accurate and consistent with the claim? The temporal information should be verifiable through the evidence.
4. **Manipulation Subtlety:** For negative samples, is the manipulation subtle enough to challenge models? Obvious or trivial manipulations are flagged for removal.

▷ *Verification Results:* 98.6% of the sampled instances are accepted, indicating high quality of the automated pipeline.

These results demonstrate that our automated construction pipeline produces high-quality benchmark samples, with the vast majority meeting human standards for factuality, evidence sufficiency, and manipulation subtlety.

B.4. RealFact Benchmark Examples

Figure 3 presents two representative examples from the RealFact benchmark, illustrating a fake sample and a real sample.



Figure 3. Representative examples from the RealFact benchmark. (a) A fake sample with manipulated content. (b) A real sample with verified factual content.

C. Details of Tool Trajectory Seed Guidance

This section provides additional details for the Tree-Structured Self-Evolving Tool Trajectory Generation process described in Sec. 3. Specifically, we elaborate on four key components: (1) the *Guider* judgment criteria used during seed trajectory construction (Sec. C.1); (2) the two-stage quality checking process for filtering high-quality seed trajectories (Sec. C.2); (3) the *Guider*’s dual role in weakly-hinted hard trajectory synthesis, where it both judges tool steps and provides weak hints to guide the Explorer (Sec. C.3); and (4) the *Refiner* that eliminates explicit hint traces to produce autonomous-looking trajectories (Sec. C.4).

C.1. Guider Judgment in Seed Trajectory Construction

During the tree-structured seed trajectory construction, the *Guider* plays a critical role in evaluating each candidate tool step. At each step t , given the current history $\{(r_i, u_i, o_i)\}_{i=1}^{t-1}$ and the final answer a^* , the *Guider* assesses whether the candidate step (r_t, u_t, o_t) is valid and helpful for solving the task. The *Guider* judges the step based on whether the selected tool is appropriate for the current reasoning state, whether the tool parameters are reasonable, whether the returned observation provides useful information, and whether the step advances the reasoning toward the correct answer. If the step is judged invalid, the branch is immediately pruned; otherwise, the step is retained for further expansion.

The detailed prompt used for the *Guider* judgment is provided below.

Prompt for Guider Judgment

System Instruction: You are an expert Guider assisting an Explorer agent in constructing high-quality tool-use trajectories for vision-language forensics tasks. You have access to the ground truth answer a^* .

Your Task: Evaluate whether the candidate tool step is valid and helpful for solving the task. Consider the following criteria:

1. **Tool Relevance:** Is the selected tool appropriate for the current task and reasoning state?
2. **Parameter Quality:** Are the tool parameters (e.g., search query, crop box, zoom region) specific and reasonable?
3. **Observation Utility:** Does the returned observation provide useful, non-redundant information?
4. **Progress Toward Answer:** Does this step bring the reasoning closer to the ground truth answer a^* ?

Input Data: Task Question: {question} **Ground Truth Answer:** a^* **Current History:** $\{(r_i, u_i, o_i)\}_{i=1}^{t-1}$ **Candidate Step:** (r_t, u_t, o_t)

Output Format: Return strictly in JSON format: { "judgment": "Valid" or "Invalid", "reason": "Brief explanation of the judgment." }

C.2. Quality Checking Process

After the *Explorer* completes the search and outputs the final prediction $\hat{a}$, we apply a two-stage quality checking process to ensure the trajectory meets the required standards.

▷ *Stage 1: Outcome-Based Filtering:* We first filter trajectories based on the correctness of the final prediction $\hat{a}$:

- **Classification Tasks:** Trajectories with incorrect final answers are removed. Only trajectories where $\hat{a}$ matches the ground truth label a^* are retained.
- **Image Grounding Tasks:** We compute the Intersection over Union (IoU) between the predicted bounding box and the ground truth annotation. Only trajectories with $\text{IoU} \geq 0.7$ are retained.
- **Text Grounding Tasks:** We compute the F1 score between the predicted text span and the ground truth annotation. Only trajectories with $\text{F1} \geq 0.7$ are retained.
- **Video Grounding Tasks:** We compute the temporal Intersection over Union (tIoU) between the predicted temporal segment and the ground truth annotation. Only trajectories with $\text{tIoU} \geq 0.7$ are retained.

- **Segmentation Tasks:** We compute the Dice score between the predicted mask and the ground truth mask. Only trajectories with Dice ≥ 0.7 are retained.
- **Real-time Event Verification:** Trajectories with incorrect verification results (True/False) are removed.

▷ *Stage 2: Trajectory Quality Assessment:* For trajectories that pass Stage 1, we further assess the overall quality using the following criteria:

1. **Trajectory Coherence:** The overall trajectory should be logically coherent, with each step building upon previous steps without abrupt jumps or contradictions.
2. **Evidence Support:** The intermediate reasoning steps and tool usage should collectively support the final conclusion. The evidence gathered should directly verify or refute the claim, reveal manipulations, or localize relevant regions.
3. **No Ineffective Steps:** The trajectory should not contain redundant tool invocations, empty observations, or reasoning steps that do not contribute to the final conclusion.
4. **Completeness:** The trajectory should contain sufficient evidence to support the prediction, rather than arriving at the correct answer through lucky guessing.

The detailed prompt used for trajectory quality assessment is provided below.

Prompt for Trajectory Quality Assessment

System Instruction: You are an expert quality assessor for tool-use trajectories in vision-language forensics. Your task is to evaluate whether a trajectory meets the required quality standards.

Your Task: Assess the trajectory based on the following criteria:

1. **Coherence:** Is the trajectory logically coherent without abrupt jumps or contradictions?
2. **Evidence Support:** Do the reasoning steps and tool usage collectively support the final conclusion?
3. **Efficiency:** Does the trajectory contain ineffective or redundant steps?
4. **Completeness:** Is there sufficient evidence to support the prediction?

Input Data: **Task Question:** {question} **Ground Truth Answer:** a^* **Trajectory:** $\tau = \{(r_t, u_t, o_t)\}_{t=1}^T \cup \{\hat{a}\}$

Final Prediction: $\hat{a}$

Output Format: Return strictly in JSON format: { "quality": "Pass" or "Fail", "reason": "Brief explanation of the quality assessment." }

Only trajectories that pass both stages of the quality checking process are included in the seed set $\mathcal{D}_s^0$. This rigorous filtering ensures that the seed trajectories used for SFT are of high quality, providing a strong foundation for subsequent self-evolution and RL training.

C.3. Guider in Weakly-Hinted Hard Trajectory Synthesis

In the Weakly-Hinted Hard Trajectory Synthesis stage (Sec. 3.4), the *Guider* plays a dual role compared to the Seed Guidance stage. Beyond judging whether the current tool step is valid, the *Guider* also provides *weak hints*—necessary parameters or local clues related to the current tool invocation—to narrow down the search space for the *Explorer*. For example, in image scenarios, the *Guider* provides a rough candidate region to assist cropping or zooming; in text scenarios, it may suggest keywords or entity names for search queries. These weak hints help the *Explorer* overcome the difficulty of exploring hard samples, while keeping the hints sufficiently vague to avoid directly leaking the answer.

The detailed prompt used for the *Guider* with weak hints is provided below.

Prompt for Guider with Weak Hints

System Instruction: You are an expert Guider assisting an Explorer agent in constructing tool-use trajectories for hard samples in vision-language forensics. You have access to the ground truth answer a^* . Your role is to evaluate tool steps and provide weak hints to guide the Explorer.

Your Task: For each candidate tool step, perform two actions:

1. **Judgment:** Evaluate whether the tool step is valid based on tool relevance, parameter quality, observation utility, and progress toward the answer.
2. **Weak Hint:** If the step is invalid or suboptimal, provide a weak hint to guide the Explorer. The hint should be:
 - Sufficient to narrow down the search space (e.g., a rough region for cropping, keywords for search).
 - Vague enough to avoid directly revealing the answer a^* .
 - Focused on local clues rather than global conclusions.

Input Data: **Task Question:** {question} **Ground Truth Answer:** a^* **Current History:** $\{(r_i, u_i, o_i)\}_{i=1}^{t-1}$ **Candidate Step:** (r_t, u_t, o_t)

Output Format: Return strictly in JSON format: { "judgment": "Valid" or "Invalid", "hint": "Weak hint for the Explorer (empty if valid)", "reason": "Brief explanation of the judgment." }

C.4. Refiner for Hint Trace Elimination

After obtaining the weakly-hinted candidate trajectories, we employ a *Refiner* to eliminate explicit traces introduced by the weak hints. The *Refiner* is a randomly selected model from the MLLM expert pool $\mathcal{M}$ (distinct from the Explorer and Guider), which rewrites the reasoning process r_t at each step of the trajectory. The goal is to remove answer-aware hints from the reasoning, preventing the model from learning hint-dependent reasoning patterns that do not generalize to real inference scenarios where no hints are available.

Specifically, the *Refiner* rewrites each reasoning step to:

1. Remove explicit references to the hint (e.g., "the Guider suggested region X" $\rightarrow$ "based on visual analysis, region X appears suspicious").
2. Rephrase the reasoning to sound as if it were generated autonomously by the Explorer.
3. Preserve the factual content and logical flow of the original reasoning.

The detailed prompt used for the *Refiner* is provided below.

Prompt for Refiner (Hint Trace Elimination)

System Instruction: You are an expert Refiner responsible for rewriting reasoning processes in tool-use trajectories. Your task is to eliminate explicit traces of weak hints that were provided by a Guider, so that the final trajectory appears as if it were generated autonomously without any external guidance.

Your Task: For each reasoning step r_t in the trajectory, rewrite it to:

1. Remove any explicit references to hints (e.g., “the Guider suggested...”, “based on the hint...”).
2. Rephrase the reasoning to sound natural and autonomous, as if the Explorer discovered the information through its own analysis.
3. Preserve the factual content, logical flow, and tool invocation decisions.
4. Ensure the rewritten reasoning is consistent with the tool invocation u_t and the returned observation o_t .

Input Data: Task Question: {question} **Original Trajectory:** $\tau = \{(r_t, u_t, o_t)\}_{t=1}^T \cup \{\hat{a}\}$ **Weak Hints Used:** {hints provided by the Guider at each step}

Output Format: Return the rewritten trajectory with hint traces eliminated: $\tau' = \{(r'_t, u_t, o_t)\}_{t=1}^T \cup \{\hat{a}\}$ where r'_t is the rewritten reasoning process for step t .

All trajectories that pass the Refiner and the subsequent quality checking process (same as in Sec. C.2) are included in the hard-sample set $\mathcal{D}_{sh}$.

D. Implementation Details

We train our model on 32 NVIDIA H200 GPUs (141 GB) using Qwen3VL-8B as the backbone, and adopt AdamW throughout all training stages. In Stage I of CGARL, the SFT phase uses a batch size of 32, a learning rate of 6×10^{-5} , and 3 epochs, while the RL phase uses a batch size of 64 (i.e., 64 sampled trajectories per step), a learning rate of 1.2×10^{-6} , 2 epochs, and a KL penalty of 0.01. For the three fine-grained tasks—image tampering localization, text tampering localization, video temporal tampering localization and segmentation—the parameter α in the reward mapping function is consistently set to 3. The data scale ratio between $\mathcal{D}_{rl}^1$ and $\mathcal{D}_{rl}^2$ is approximately 7 : 3.

D.1. Reward Function Configuration

In the RL phase, the total reward R for a generated response y given question x and ground truth a^* is composed of three distinct components: the task-specific performance score (R_{task}), the structural format reward (R_{fmt}), and a repetition penalty (R_{rep}). The final reward is calculated as:

$$R(y, x) = R_{\text{task}}(y, x) + R_{\text{fmt}}(y) + R_{\text{rep}}(y) \quad (9)$$

1. Task-Specific Performance Reward (R_{task})

We employ distinct metrics for different forensic tasks. To encourage the model to strive for high-precision grounding rather than mediocre overlap, we apply an exponential mapping function to continuous metrics (IoU, F1, tIoU).

- **Binary Forgery Classification:** For the binary classification task, we use a simple discrete reward based on correctness:

$$R_{\text{task}}^{\text{cls}} = \begin{cases} 1, & \text{if } \hat{a} = a^*, \\ 0, & \text{otherwise.} \end{cases} \quad (10)$$

- **Tampering Localization (Image, Text, Video):** For fine-grained localization tasks, we compute a raw metric $m \in [0, 1]$ specific to the modality: Intersection over Union (IoU) for image bounding boxes, F1-score for text indices, and Temporal IoU (tIoU) for video segments. The raw metric is mapped to a normalized reward using an exponential function with a scaling factor $\alpha = 3$:

$$R_{\text{task}}^{\text{loc}}(m) = \frac{e^{\alpha \cdot m} - 1}{e^{\alpha} - 1} \quad (11)$$

This convex mapping ($\alpha = 3$) amplifies the reward signal for high-quality responses (e.g., $m > 0.7$) while suppressing rewards for low-quality overlaps.

- **Forgery Segmentation:** For the segmentation task, we compute a composite score that combines box-level IoU and point prompt consistency. Specifically, we first convert the predicted bounding box into a binary mask M_{pred} (1 inside the box, 0 outside) of the same size as the ground truth mask M_{gt} , and compute the Box IoU:

$$\text{IoU}_{\text{box}} = \frac{|M_{\text{pred}} \cap M_{\text{gt}}|}{|M_{\text{pred}} \cup M_{\text{gt}}|} \quad (12)$$

We then evaluate point prompt consistency: for each predicted point, we check whether its label is consistent with the ground truth mask (i.e., positive points should fall inside M_{gt} and negative points should fall outside). The point consistency score is computed as:

$$\text{score}_{\text{point}} = \frac{\text{number of consistent points}}{\text{total number of points}} \quad (13)$$

The final segmentation score is a weighted combination:

$$m_{\text{seg}} = 0.7 \times \text{IoU}_{\text{box}} + 0.3 \times \text{score}_{\text{point}} \quad (14)$$

clipped to $[0, 1]$. This composite score is then mapped through the same exponential function with $\alpha = 3$:

$$R_{\text{task}}^{\text{seg}}(m_{\text{seg}}) = \frac{e^{\alpha \cdot m_{\text{seg}}} - 1}{e^{\alpha} - 1} \quad (15)$$

2. Format Compliance Reward (R_{fmt})

To enforce the tool trajectory structure, we employ a strict regular expression check. The model receives a positive reward if and only if the output strictly follows the required format:

$$R_{\text{fmt}} = \begin{cases} 0.2 & \text{if format is valid} \\ 0.0 & \text{otherwise} \end{cases} \quad (16)$$

3. Repetition Penalty (R_{rep})

To prevent degenerate generation loops, we implement an N-gram repetition penalty. We calculate the ratio of unique N-grams to total N-grams in the generated text. With a set hyperparameters of $N = 3$ and a maximum penalty coefficient $\lambda_{\text{pen}} = -1.0$, the penalty is defined as:

$$R_{\text{rep}} = \lambda_{\text{pen}} \times \left(1 - \frac{|S_{\text{unique}}^N|}{N_{\text{total}}} \right) \quad (17)$$

where $|S_{\text{unique}}^N|$ is the count of unique 3-grams and N_{total} is the total number of 3-grams. This term introduces a negative reward proportional to the redundancy of the text.

D.2. Advantage Weighting Function

The token-level weighting function $f_{i,t}(\cdot)$ in Eq. (6) follows the SAPO design (Gao et al., 2025). Specifically, for each response y_i with normalized advantage $\hat{A}_{i,k}$, the weighting function is defined as:

$$f_{i,t}^{\text{safo}}(r_{i,t}(\theta)) = \frac{4}{\tau_i} \sigma(\tau_i(r_{i,t}(\theta) - 1)), \quad \tau_i = \begin{cases} \tau_{\text{pos}}, & \hat{A}_{i,k} > 0, \\ \tau_{\text{neg}}, & \hat{A}_{i,k} \leq 0, \end{cases} \quad (18)$$

where $r_{i,t}(\theta) = \frac{\pi_{\theta}(y_{i,t}|x, y_{i,<t})}{\pi_{\theta_{\text{old}}}(y_{i,t}|x, y_{i,<t})}$ is the probability ratio, τ_{neg} and τ_{pos} denote the temperature parameters for positive and negative tokens, respectively, and $\sigma(x) = \frac{1}{1+e^{-x}}$ represents the sigmoid function.

D.3. Hyperparameter settings in ARSPO

Table 1. Hyperparameter settings for the Dynamic Coefficient Adjustment Algorithm in ARSPO.

Symbol	Description	Value
<i>Schedule & Initialization</i>		
T_{warm}	Warm-up phase duration (steps)	800
T	Coefficient update interval (steps)	100
<i>Adjustment Dynamics</i>		
α_{boost}	Coefficient boost factor	1.1
α_{decay}	Coefficient decay factor	0.9
<i>Thresholds</i>		
ϵ_{mom}	Momentum protection threshold	0.02
ϵ_{rescue}	Regression rescue threshold	0.10
$\mathcal{T}_{high}$	High-performance threshold	0.60

E. Inference Protocol for Tool Interaction

At inference time, OmniVL-Guard Pro is executed as an interactive tool-augmented agent rather than a one-shot trajectory generator. Given an input question x , the policy first generates a reasoning segment r_t and, when external clues or fine-grained inspection is needed, emits a structured tool invocation u_t following the predefined action format. The environment parses u_t , executes the corresponding tool in $\mathcal{T}$, and returns the observation o_t , such as retrieved evidence, a cropped or zoomed image region, detected face boxes, extracted video frames, or a SAM3 segmentation mask. The observation o_t is then appended to the context, and the policy continues generation conditioned on the updated interaction history $\{(r_i, u_i, o_i)\}_{i=1}^t$. This reasoning–action–observation loop is repeated until the policy emits a final answer $\hat{a}$ or reaches the maximum interaction budget.

Formally, for each test sample x , inference proceeds as

$$r_t, u_t \sim \pi_\theta(\cdot \mid x, \{(r_i, u_i, o_i)\}_{i=1}^{t-1}), \quad o_t = \text{Env}(u_t),$$

where $\text{Env}(\cdot)$ denotes the tool environment described in Sec. 2. The final prediction is produced only after the model has incorporated the observations returned by its previous tool calls:

$$\hat{a} \sim \pi_\theta(\cdot \mid x, \{(r_i, u_i, o_i)\}_{i=1}^T).$$

Therefore, the predicted trajectory is not merely a textual explanation generated after the fact; it records the actual sequence of tool invocations and observations used during inference.

We use the same tool interfaces, action schema, and observation format as in trajectory construction and RL training. To avoid unbounded exploration, inference is constrained by a maximum number of tool calls $T_{\max}$. If a generated tool invocation is malformed or refers to an unavailable tool, the environment returns an error observation and the model is allowed to continue, subject to the same interaction budget. Please refer to Appendix J for examples.

F. Details of Checker Construction

This section provides additional details for the Checker construction process described in Sec. 4.2. Specifically, we elaborate on the scoring criteria for the discrete three-level score $s \in \{0, 0.5, 1\}$ and the prompts used for MLLM-based evaluation and verification.

F.1. Scoring Criteria for Process Quality

The Checker assigns a discrete score $s \in \{0, 0.5, 1\}$ to evaluate the quality of the tool reasoning trajectory. The detailed criteria for each score level are as follows:

Score $s = 1$: Fully Consistent and Sufficient. A trajectory receives the highest score when it satisfies all of the following conditions:

- **Internal Consistency:** The reasoning steps form a logically coherent chain without contradictions or abrupt jumps. Each step builds upon previous observations and maintains a clear narrative flow.
- **Reasonable Tool Usage:** The selected tools are appropriate for the current task and reasoning state. Tool parameters (e.g., search queries, crop boxes, zoom regions) are specific and well-justified. The model demonstrates understanding of when and how to invoke each tool.
- **Sufficient Evidence:** The acquired observations collectively provide sufficient evidence to support the final prediction. The evidence chain is complete, with no critical gaps or missing links that would undermine the conclusion.
- **Prediction-Process Alignment:** The final prediction $\hat{a}$ is clearly and directly supported by the accumulated evidence and reasoning. There is no contradiction between the reasoning process and the final answer.

Score $s = 0.5$: Partially Consistent with Limitations. A trajectory receives a moderate score when it exhibits local correctness but suffers from one or more of the following issues:

- **Insufficient Evidence:** While the reasoning is locally sound, the trajectory fails to gather enough evidence to conclusively support the prediction. Some tool calls may return uninformative observations, or the model may prematurely conclude without exploring all relevant aspects.
- **Incomplete Intermediate Support:** The trajectory contains reasonable steps, but some intermediate conclusions are not fully supported by the preceding evidence. There may be minor logical gaps or assumptions that are not adequately validated.
- **Redundant but Non-Detracting Steps:** The trajectory includes some redundant tool calls or reasoning steps that do not contribute to the final conclusion, but do not actively mislead the reasoning process.
- **Weak Prediction Alignment:** The final prediction is plausible given the evidence, but the connection between the reasoning and the conclusion is not explicitly articulated or relies on implicit assumptions.

Score $s = 0$: Inconsistent or Insufficient. A trajectory receives the lowest score when it exhibits one or more of the following critical issues:

- **Logical Inconsistency:** The reasoning chain contains contradictions, where later steps contradict earlier observations or conclusions. The trajectory lacks a coherent narrative structure.
- **Inappropriate Tool Usage:** The model selects inappropriate tools for the task, provides unreasonable parameters, or fails to interpret tool outputs correctly. The tool calls do not contribute meaningfully to the reasoning process.
- **Missing Evidence Chain:** The trajectory lacks a self-consistent evidence chain. Critical evidence is missing, and the final prediction is not supported by the available observations.
- **Prediction-Process Mismatch:** The final prediction $\hat{a}$ is clearly inconsistent with the reasoning process. The model may arrive at the correct answer through lucky guessing or flawed reasoning, which constitutes a pseudo-success pattern.
- **Format Violations:** The trajectory exhibits severe format issues, such as missing tool invocations, incomplete responses, or malformed output structures.

F.2. MLLM Prompts for Checker Construction

We provide the prompts used for the MLLM-based evaluation and verification in the Checker construction process.

Prompt for Generating Score and Rationale. The following prompt is used by the two evaluation MLLMs to assess the trajectory and generate both the fine-grained evaluation rationale and the discrete score s :

Prompt for Evaluation MLLM (Score and Rationale Generation)

System Instruction: You are an expert evaluator for tool-use trajectories in vision-language forensics. Your task is to assess the quality of a reasoning trajectory that uses external tools to verify the authenticity of content. You must provide a detailed evaluation rationale and assign a discrete score based on the criteria below.

Scoring Criteria:

Score 1 (Fully Consistent and Sufficient): The trajectory satisfies all of the following: (a) Internal Consistency—reasoning steps form a logically coherent chain without contradictions or abrupt jumps; (b) Reasonable Tool Usage—selected tools are appropriate for the task and reasoning state, and parameters are specific and well-justified; (c) Sufficient Evidence—acquired observations collectively provide sufficient evidence to support the final prediction, with no critical gaps; (d) Prediction-Process Alignment—the final prediction $\hat{a}$ is clearly and directly supported by the accumulated evidence and reasoning.

Score 0.5 (Partially Consistent with Limitations): The trajectory exhibits local correctness but suffers from one or more of the following: (a) Insufficient Evidence—reasoning is locally sound but fails to gather enough evidence to conclusively support the prediction; (b) Incomplete Intermediate Support—some intermediate conclusions are not fully supported by the preceding evidence; (c) Redundant but Non-Detracting Steps—some redundant tool calls or reasoning steps that do not contribute to the final conclusion but do not actively mislead; (d) Weak Prediction Alignment—the final prediction is plausible but the connection between reasoning and conclusion is not explicitly articulated.

Score 0 (Inconsistent or Insufficient): The trajectory exhibits one or more critical issues: (a) Logical Inconsistency—reasoning chain contains contradictions where later steps contradict earlier observations or conclusions; (b) Inappropriate Tool Usage—inappropriate tool selection, unreasonable parameters, or incorrect interpretation of tool outputs; (c) Missing Evidence Chain—critical evidence is missing and the final prediction is not supported by the available observations; (d) Prediction-Process Mismatch—the final prediction is clearly inconsistent with the reasoning process (pseudo-success pattern); (e) Format Violations—severe format issues such as missing tool invocations, incomplete responses, or malformed output structures.

Input Data: Task Question: {question} **Ground Truth Answer:** a^* **Trajectory:** $\tau = \{(r_t, u_t, o_t)\}_{t=1}^T \cup \{\hat{a}\}$

Final Prediction: $\hat{a}$

Output Format: Return strictly in JSON format: { “evaluation”: “Detailed analysis of the trajectory quality. Specifically address: (1) whether the reasoning steps are internally consistent, (2) whether the tool usage is appropriate and well-parameterized, (3) whether the acquired evidence is sufficient to support the conclusion, and (4) whether the final prediction aligns with the reasoning process.”, “score”: 0, 0.5, or 1 }

Prompt for Verification. After an evaluation MLLM generates its evaluation and score, a third MLLM from the pool $\mathcal{M}$ independently verifies each evaluation. Specifically, for each evaluation-scorepair produced by an evaluation MLLM, the verification MLLM checks whether the evaluation is reasonable and the score is justified. The verification is performed one evaluation at a time, and the verification prompt is as follows:

Prompt for Verification MLLM (Single Evaluation Verification)

System Instruction: You are a quality assurance verifier for tool-use trajectory evaluations. An evaluator has assessed a trajectory and provided an evaluation rationale along with a score. Your task is to verify whether the evaluation is reasonable, accurate, and well-justified.

Verification Criteria:

- Check whether the evaluation accurately reflects the trajectory quality according to the scoring criteria.
- Identify any factual errors in the evaluation (e.g., incorrect claims about tool usage or evidence).
- Assess whether the evidence cited in the evaluation is accurate and relevant to the trajectory.
- Determine if the score is justified given the evaluation rationale.

Input Data: **Task Question:** {question} **Ground Truth Answer:** a^* **Trajectory:** $\tau = \{(r_t, u_t, o_t)\}_{t=1}^T \cup \{\hat{a}\}$
Final Prediction: $\hat{a}$ **Evaluation:** {evaluation from evaluation MLLM} **Score:** {score from evaluation MLLM}
Output Format: Return strictly in JSON format: { "verification": "Detailed analysis of whether the evaluation is reasonable and accurate. Highlight any factual errors or inconsistencies found.", "is_valid": true or false, "reason": "Explanation for the verification decision. If invalid, specify which aspects of the evaluation contain errors." }

All samples that pass the verification process, where both evaluation MLLMs agree on the score and the verification MLLM confirms the validity, are included in the Checker training set $\mathcal{D}_{\text{checker}}$.

G. Analysis of Self-Evolution Iteration Rounds

To determine the optimal termination point for the Tree-Structured Self-Evolving Tool Trajectory Generation strategy (Sec. 3.3), we investigated the performance trajectory across iterations n . We hypothesized that the self-evolution process would eventually reach a saturation point where the teacher policy π_n no longer provides superior tool-use signals compared to π_{n-1} , thereby yielding no further quality improvements in the generated dataset $\text{FSTR}_{\text{sft}}^n$.

Specifically, we compared the capabilities of the model trained after the fourth iteration against a model extended to a fifth iteration. We constructed a fifth-round dataset $\text{FSTR}_{\text{sft}}^5$ using π_4 and combined it with the hard-sample set $\mathcal{D}_{sh}$ to train the full OmniVL-Guard Pro framework via CGARL. We then compared this against our standard setting ($\text{FSTR}_{\text{sft}}^4 \cup \mathcal{D}_{sh}$).

Table 2. Performance comparison between stopping at Iteration 4 ($\text{FSTR}_{\text{sft}}^4$) vs. Iteration 5 ($\text{FSTR}_{\text{sft}}^5$). The results indicate that performance plateaus at $n = 4$, justifying the termination of the self-evolution loop to conserve computational resources.

Training Set	Binary Classification (ACC)					Localization & Segmentation			
	Text	Image	Video	Text-Image	RealFact	Image (IoU)	Text (F1)	Video (tIoU)	Seg. (Dice)
$\text{FSTR}_{\text{sft}}^4 \cup \mathcal{D}_{sh}$ (Ours)	97.38	94.67	99.03	78.91	86.45	58.72	66.94	64.37	55.90
$\text{FSTR}_{\text{sft}}^5 \cup \mathcal{D}_{sh}$	97.21	94.89	98.98	79.15	86.34	58.55	67.23	64.19	56.01
$\text{Diff}(\Delta)$	+0.17	-0.22	+0.05	-0.24	+0.11	+0.17	-0.29	+0.18	-0.11

As illustrated in Table 2, the performance differences between the fourth and fifth iterations are statistically negligible. For instance, the Image Localization (IoU) shifted marginally from 58.72 to 58.55 ($\Delta = +0.17$), while Text Localization (F1) experienced a slight fluctuation from 66.94 to 67.23 ($\Delta = -0.29$). The results show a mix of minor improvements and degradations across tasks, with no consistent gains. This evidence suggests that the quality of the tool trajectory data generated by π_4 has reached a bottleneck, and further iterations do not yield distinguishable gains in downstream forensic tasks. Consequently, we terminate the self-evolution process at $n = 4$ to strike an optimal balance between model performance and training efficiency.

H. Ablation Study on Weakly-Hinted Hard Trajectory Synthesis

To validate the necessity of the **Weakly-Hinted Hard Trajectory Synthesis** stage (Sec. 3.4), we conducted an ablation study by excluding the hard-sample subset $\mathcal{D}_{sh}$ from the SFT cold-start dataset.

Experimental Settings:

- **Full Setting (Ours):** The model is fine-tuned on the complete dataset $\text{FSTR}_{\text{sft}} = \mathcal{D}_s^4 \cup \mathcal{D}_{sh}$, followed by CGARL training.
- **w/o Hard Samples:** The model is fine-tuned only on the self-evolved dataset $\mathcal{D}_s^4$ (removing the synthesized hard samples $\mathcal{D}_{sh}$), followed by the same CGARL training.

The results are reported in Table 3. We observe that removing the hard samples leads to a consistent performance degradation across all tasks.

Table 3. Ablation study on the necessity of Weakly-Hinted Hard Trajectory Synthesis. We report the performance comparison between using the full dataset versus removing the synthesized hard samples ($\mathcal{D}_{sh}$). The results show that $\mathcal{D}_{sh}$ is critical for maintaining high performance across both classification and localization tasks.

Method	Binary Classification (ACC)					Localization & Segmentation			
	Text	Image	Video	Text-Image	RealFact	Image (IoU)	Text (F1)	Video (tIoU)	Seg. (Dice)
Ours (Full FSTR_{sft})	97.38	94.67	99.03	78.91	86.45	58.72	66.94	64.37	55.90
w/o Hard Samples ($\mathcal{D}_{sh}$)	93.45	88.72	92.18	71.56	82.23	52.84	57.31	56.42	48.23
Performance Drop (Δ)	-3.93	-5.95	-6.85	-7.35	-4.22	-5.88	-9.63	-7.95	-7.67

Analysis of Results:

- **Impact on Fine-grained Localization and Segmentation:** The performance drop is significant in localization and segmentation tasks. Specifically, Text Localization (F1) drops by **9.63%**, Video Temporal Localization (tIoU) drops by **7.95%**, and Segmentation (Dice) drops by **7.67%**. This empirical evidence suggests that the self-evolution loop alone tends to saturate on easier samples. The failure cases (hard samples) from the iterative process often contain subtle manipulations or complex multimodal inconsistencies that require the specific, high-quality tool-use trajectories provided by the Weakly-Hinted Hard Trajectory Synthesis to resolve.
- **Impact on Binary Classification:** The binary classification accuracy also decreases notably (e.g., **-7.35%** on Text-Image). This indicates that the hard-sample data is indispensable not only for fine-grained grounding and segmentation but also for maintaining robust detection capabilities in complex scenarios.

In conclusion, the Weakly-Hinted Hard Trajectory Synthesis effectively complements the self-evolution pipeline by covering the long-tail distribution of difficult forgery types, preventing the model from overfitting to simple patterns.

I. Limitations

I.1. Computational Cost and Open-Source Commitment

Both the construction of the FSTR dataset and the multi-stage training of OmniVL-Guard Pro demand substantial computational resources, including large-scale tool-environment rollouts, multi-iteration self-evolution, and multi-stage RL optimization. Our goal is to establish a foundation model for unified vision-language forensics that supports diverse downstream tasks. To lower the barrier for the community and facilitate future research, we commit to fully open-sourcing the FSTR dataset, the trained OmniVL-Guard Pro checkpoints, and all associated code, enabling researchers to directly fine-tune or evaluate on their own tasks without reproducing the expensive training pipeline.

I.2. Safeguards Against Misuse

Given that the RealFact benchmark, FSTR dataset, and OmniVL-Guard Pro model could potentially be repurposed for malicious activities such as generating or evading forgery detection, we will implement a strict access control process. All assets will be released exclusively for academic research purposes, and each applicant will be required to provide institutional affiliation and a clear statement of intended use. Access will be granted only after a manual review, and any violation of the terms will result in immediate revocation.

J. Case Study

Case1 - Text-Image Forgery Detection

Question: Please determine if the image is **Real**, **AI-generated**, or **Tampered**.



Caption: Braun poses with Hall of Famer Robin Yount and his 2011 National League MVP award.



Tampered



OmniVL-Guard Pro:

<think>Okay, let's see. The user provided a news image with a caption saying Braun poses with Hall of Famer Robin Yount and his 2011 National League MVP award. I need to check if this is real, AI-generated, or tampered.

First, I should verify the event. The caption mentions Ryan Braun winning the 2011 NL MVP. I know that Ryan Braun did win the MVP in 2011, so that part is correct. But the image shows him with Robin Yount, who is a Hall of Famer. Wait, Robin Yount is a former Milwaukee Brewers player, and he was inducted into the Hall of Fame in 1999. So the context here might be plausible.

But maybe there's something off. Let me think. The user wants to check if the image is real or tampered. Since it's a photo, possible issues could be deepfake, editing, or incorrect elements.

The tools available include `detect_edge_anomaly`, which can find possible tampering. But first, since it's a single image (not a video), I don't need to use the video extraction tool. The image index is 0 (since it's the first image provided).

So, maybe run `detect_edge_anomaly` on the image to see if there are any suspicious regions. If there are anomalies, then it might be tampered. If not, maybe it's real. Also, check faces with `detect_faces` to see if the people match known individuals.

Wait, the caption mentions Robin Yount. Let me confirm: Robin Yount is a real person, a Hall of Famer. Ryan Braun is also real. The 2011 MVP award is accurate. So the content seems plausible, but the image could still be edited.

Let me proceed step by step. First, check for edge anomalies. Call `detect_edge_anomaly` on `image_index 0`. That might highlight areas where the image was altered. If the tool returns some regions, then investigate those further with `zoom` or `crop`. If no anomalies, maybe the image is real.

So the first tool call should be `detect_edge_anomaly` with `image_index 0`. **</think>**

<tool_call>`{"name": "detect_edge_anomaly", "arguments": {"image_index": 0}}`**</tool_call>**

<tool_response>`{"status": "success", "images": "<image>", "output": {"suspicious_boxes": [{"score": 2.6052839756011963, "box": [188, 784, 312, 889], "edge_density": 0.294921875, "lap_energy": 73.44140625, "neighbor_gap": 8.88311767578125}, {"score": 2.3950839042663574, "box": [188, 523, 312, 627], "edge_density": 0.3505859375, "lap_energy": 77.76953125, "neighbor_gap": 3.6822509765625}, {"score": 2.363237142562866, "box": [188, 575, 312, 680], "edge_density": 0.3515625, "lap_energy": 76.7265625, "neighbor_gap": 3.54803466796875}, {"score": 2.3256735801696777, "box": [188, 627, 312, 732], "edge_density": 0.35546875, "lap_energy": 74.46484375, "neighbor_gap": 3.46331787109375}, {"score": 2.3091726303100586, "box": [188, 418, 312, 523], "edge_density": 0.3408203125, "lap_energy": 75.4140625, "neighbor_gap": 3.7838134765625}], "global_stats": {"block_size": 32, "stride": 16, "canny_low": 60, "canny_high": 160, "num_windows_y": 18, "num_windows_x": 15, "max_score": 2.6052839756011963, "mean_score": 0.20154379308223724}}}`**</tool_response>**



<think>The edge anomaly detection highlighted several suspicious regions on the left person's shirt (vertical stripes). These anomalies could indicate digital tampering, such as inconsistent edge patterns or blending artifacts. To investigate further, I should zoom into one of these high-score regions to examine the texture and details. The highest-scoring box is [188, 784, 312, 889], which corresponds to the lower part of the striped shirt. Magnifying this area with a `scale` of 3.0 will help determine if the pattern shows signs of manipulation like unnatural repetition or blurring. **</think>**

<tool_call>`{"name": "zoom_image", "arguments": {"image_index": 0, "x1": 188, "y1": 784, "x2": 312, "y2": 889, "scale": 3.0}}`**</tool_call>**



<tool_response>`{"status": "success", "images": "<image>", "output": {"box": [188, 784, 312, 889], "scale": 3.0}}`**</tool_response>**



<think>The zoomed-in view of the suspicious region shows highly distorted and inconsistent stripe patterns with unnatural blurring and pixelation. This suggests digital tampering, likely involving AI-based editing to alter the original image. The irregularities in the fabric texture are not typical of natural photography artifacts. Given the edge anomaly results combined with the visual inconsistencies in the magnified region, the image appears to be tampered with. **</think>**

<answer>Answer: Tampered**</answer>**

Case2 - Text Tampering Location

Question: Please locate the tampered words in the caption and provide the indices of the altered words as a list (starting from 0).



Caption: John Swinney expressed that he was *in the depths of despair* but was doing everything he currently could to support the economy.



[6, 8, 9, 10, 11]



OmniVL-Guard Pro:



<think>I need to analyze whether any words in the caption are inconsistent with the image content. First, I'll break down the caption into individual words with their indices for reference:

0: John 1: Swinney 2: expressed 3: that 4: he 5: was 6: in 7: the 8: depths 9: of 10: despair 11: but 12: was 13: doing 14: everything 15: he 16: currently 17: could 18: to 19: support 20: the 21: economy

The image shows a man holding documents labeled "SCOTTISH BUDGET," which suggests a formal economic context. The phrase "in the depths of despair" stands out as emotionally intense and potentially inconsistent with the professional tone expected in a budget presentation. I'll start by examining the image for any visual anomalies that might support or contradict the caption's emotional tone. I'll use detect_edge_anomaly to check for manipulated regions, especially around facial features or text elements.

<tool_call> {"name": "detect_edge_anomaly", "arguments": {"image_index": 0}} **</tool_call>**

<tool_response> {"status": "success", "images": "<image>", "output": {"suspicious_boxes": [{"score": 62853.09375, "box": [656, 70, 688, 94], "edge_density": 0.20703125, "lap_energy": 23.193359375, "neighbor_gap": 3.9722900390625}, {"score": 61372.89453125, "box": [656, 82, 688, 106], "edge_density": 0.2021484375, "lap_energy": 25.552734375, "neighbor_gap": 4.49603271484375}, {"score": 57216.84375, "box": [812, 821, 844, 845], "edge_density": 0.1884765625, "lap_energy": 25.056640625, "neighbor_gap": 0.88128662109375}, {"score": 56920.2734375, "box": [781, 845, 812, 868], "edge_density": 0.1875, "lap_energy": 19.703125, "neighbor_gap": 2.0367431640625}, {"score": 55436.93359375, "box": [797, 833, 828, 856], "edge_density": 0.1826171875, "lap_energy": 19.482421875, "neighbor_gap": 1.3216552734375}], "global_stats": {"block_size": 32, "stride": 16, "canny_low": 60, "canny_high": 160, "num_windows_y": 84, "num_windows_x": 63, "max_score": 62853.09375, "mean_score": 4072.0302734375}}}} **</tool_response>**



<think>The detect_edge_anomaly tool identified suspicious regions near the top-right (around the forehead/hairline) and bottom-right (on the documents). The highest-scoring anomaly is at [656,70,688,94], which may relate to facial features, and another at [781,845,812,868] on the documents.

The document text "SCOTTISH BUDGET" is clearly visible, confirming the context is a formal economic announcement. The emotional phrase "in the depths of despair" seems out of place here — budget presentations typically involve measured, factual language, not expressions of personal despair. I'll now zoom into the document area to verify if the text has been altered or if there's any visual evidence supporting the caption's tone. I'll use zoom_image on the document region to examine the text more closely.

<tool_call> {"name": "zoom_image", "arguments": {"image_index": 0, "x1": 700, "y1": 750, "x2": 900, "y2": 950, "scale": 3.0}} **</tool_call>**

<tool_response> {"status": "success", "images": "<image>", "output": {"box": [700, 750, 900, 950], "scale": 3.0}} **</tool_response>**



<think>The zoomed image confirms the document reads "SCOTTISH BUDGET 2024-25," which aligns with John Swinney's role as Scotland's First Minister presenting the budget. This reinforces that the context is professional and economic, not emotional. The phrase "in the depths of despair" (indices 6,8,9,10) is highly inconsistent with this setting. Additionally, the word "but" (index 11) creates a logical contradiction — it's odd to say someone is "in the depths of despair" while simultaneously "doing everything he currently could to support the economy." The key issue remains the caption's tone versus the image's context. The man in the image has a serious but composed expression, not one of despair. The phrase "in the depths of despair but" (indices 6,8,9,10,11) is the most likely tampered section — it's emotionally exaggerated and logically inconsistent with a budget presentation. Words like "expressed" and "that" (indices 2,3) are plausible in this context and likely unchanged. Based on the visual and contextual mismatch, I conclude the tampered words are at indices [6,8,9,10,11].

<answer>Answer: [6, 8, 9, 10, 11] **</answer>**

Case3 - Image Tampering Location

Question: Please locate the tampered region in the image.



Caption: Clint Dempsey leads the celebrations after Seattle's US Open Cup final victory.

← [455, 195, 575, 449]



OmniVL-Guard Pro:



<think>The image shows a soccer match with players wearing Germany and another team's jerseys, while the caption references Clint Dempsey and Seattle's US Open Cup final — a clear mismatch. This discrepancy suggests the image may have been altered. I'll start by examining the central area where facial features appear, as that's often a target for manipulation. I'll zoom into that region to inspect for unnatural distortions or inconsistencies. **</think>**



<tool_call>{"name": "zoom_image", "arguments": {"image_index": 0, "x1": 455, "y1": 195, "x2": 575, "y2": 449, "scale": 3.0}} **</tool_call>**

<tool_response>{"status": "success", "images": "<image>", "output": {"box": [455, 195, 575, 449], "scale": 3.0}} **</tool_response>**



<think>After zooming in, I notice a facial region that appears unnaturally stretched and misaligned with surrounding elements like the soccer ball and jersey. The contours and proportions don't match typical human anatomy, which raises suspicion of digital tampering. To further investigate, I'll run edge anomaly detection on the full image to see if automated analysis confirms irregularities in this area. **</think>**

<tool_call>{"name": "detect_edge_anomaly", "arguments": {"image_index": 0}} **</tool_call>**



<tool_response>{"status": "success", "images": "<image>", "output": {"suspicious_boxes": [{"score": 10.478888511657715, "box": [529, 500, 611, 625], "edge_density": 0.259765625, "lap_energy": 80.658203125, "neighbor_gap": 12.66790771484375}, {"score": 9.203973770141602, "box": [489, 438, 570, 562], "edge_density": 0.244140625, "lap_energy": 73.990234375, "neighbor_gap": 9.82305908203125}, {"score": 9.034083366394043, "box": [529, 562, 611, 688], "edge_density": 0.234375, "lap_energy": 70.841796875, "neighbor_gap": 10.41748046875}, {"score": 8.233757972717285, "box": [570, 688, 651, 812], "edge_density": 0.251953125, "lap_energy": 59.744140625, "neighbor_gap": 8.01226806640625}, {"score": 8.169635772705078, "box": [529, 625, 611, 750], "edge_density": 0.232421875, "lap_energy": 63.3515625, "neighbor_gap": 8.4432373046875}], "global_stats": {"block_size": 32, "stride": 16, "canny_low": 60, "canny_high": 160, "num_windows_y": 15, "num_windows_x": 23, "max_score": 10.478888511657715, "mean_score": 1.0289173126220703}}} **</tool_response>**

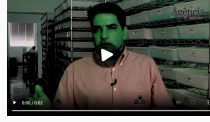


<think>The edge anomaly detection highlights several high-scoring boxes clustered around the region I previously zoomed into. The top-scoring box [529, 500, 611, 625] overlaps significantly with the area showing distorted facial features, reinforcing the presence of manipulation. Additionally, the mismatch between the caption (Clint Dempsey, Seattle, US Open Cup) and the visual content (Germany jersey, World Cup-style ball) further supports that this image has been altered. Based on both visual inspection and anomaly detection, the tampered region is confirmed. **</think>**

<answer>Answer: [455, 195, 575, 449]. **</answer>**

Case4 - Video Tampering Location

Question: Please identify the specific time segments that have been tampered with.



← ["0.80s-1.20s"]



OmniVL-Guard Pro:



<think>I need to identify tampered time segments in the video. The video shows a person in a room with shelves. At around 0.8s, there's a noticeable change where the person seems to be holding something different. Let me extract frames from a short segment around this time to analyze further. I'll start by extracting frames from 0.5s to 1.5s to cover the area where the change appears to occur.</think>

<tool_call>{"name": "extract_video_segment_frame", "arguments": {"start_sec": 0.5, "end_sec": 1.5}}</tool_call>

<tool_response>{"status": "success", "images": "<image><image><image><image>", "output": {"time_range_sec": [0.5, 1.5], "raw_frame_range": [10, 30], "raw_frame_count_in_range": 21, "sampled_frame_count_for_mllm": 4, "sampled_frame_indices_for_mllm": [10, 17, 23, 30], "sampled_frame_timestamps_sec_for_mllm": [0.5, 0.85, 1.15, 1.5], "sampled_fps_for_mllm": 3.809524, "sampled_image_count": 4}}</tool_response>



<think>The extraction returned 4 frames from 0.5s to 1.5s. The sampled timestamps are 0.5s, 0.85s, 1.15s, and 1.5s. Looking at these frames, the most noticeable change occurs between 0.85s and 1.15s where the object the person is holding changes from a flat item to a different shape. To check for tampering, I should run edge anomaly detection on the frames around this transition. I'll start with the frame at 0.85s (image_index 1) since it's near the middle of the suspected change.</think>

<tool_call>{"name": "detect_edge_anomaly", "arguments": {"image_index": 1}}</tool_call>



<tool_response>{"status": "success", "images": "<image>", "output": {"suspicious_boxes": [{"score": 2.794658660888672, "box": [943, 104, 1000, 208], "edge_density": 0.169921875, "lap_energy": 44.541015625, "neighbor_gap": 7.452148914337158}, {"score": 2.701683759689331, "box": [943, 312, 1000, 416], "edge_density": 0.177734375, "lap_energy": 40.744140625, "neighbor_gap": 7.488965034484863}, {"score": 2.5701098442077637, "box": [600, 779, 657, 883], "edge_density": 0.18359375, "lap_energy": 34.578125, "neighbor_gap": 8.0152587890625}, {"score": 2.5143895149230957, "box": [286, 260, 343, 364], "edge_density": 0.1796875, "lap_energy": 43.201171875, "neighbor_gap": 5.49493408203125}, {"score": 2.0997695922851562, "box": [257, 260, 314, 364], "edge_density": 0.212890625, "lap_energy": 36.857421875, "neighbor_gap": 2.8558349609375}], "global_stats": {"block_size": 32, "stride": 16, "canny_low": 60, "canny_high": 160, "num_windows_y": 18, "num_windows_x": 34, "max_score": 2.794658660888672, "mean_score": 0.08402005583047867}}}</tool_response>



<think>The edge anomaly detection on image_index 1 (0.85s) shows several suspicious boxes, particularly around the logo area and some background elements. However, the logo area might be a natural high-edge region. The most relevant anomaly for tampering would likely be related to the object the person is holding, which changed between frames. Let me check the next frame at 1.15s (image_index 2) to see if there's a similar anomaly in the object region. I'll run edge anomaly detection on image_index 2 to compare.</think>

<tool_call>{"name": "detect_edge_anomaly", "arguments": {"image_index": 2}}</tool_call>



<tool_response>{"status": "success", "images": "<image>", "output": {"suspicious_boxes": [{"score": 2.9673092365264893, "box": [943, 312, 1000, 416], "edge_density": 0.19921875, "lap_energy": 41.36328125, "neighbor_gap": 7.39882755279541}, {"score": 2.7297520637512207, "box": [943, 104, 1000, 208], "edge_density": 0.162109375, "lap_energy": 44.736328125, "neighbor_gap": 6.869726657867432}, {"score": 2.5150146484375, "box": [286, 260, 343, 364], "edge_density": 0.1767578125, "lap_energy": 42.673828125, "neighbor_gap": 5.3355712890625}, {"score": 2.356698513031006, "box": [600, 831, 657, 935], "edge_density": 0.197265625, "lap_energy": 36.07421875, "neighbor_gap": 5.0654296875}, {"score": 2.223613739013672, "box": [686, 0, 743, 104], "edge_density": 0.205078125, "lap_energy": 32.783203125, "neighbor_gap": 4.711621284484863}], "global_stats": {"block_size": 32, "stride": 16, "canny_low": 60, "canny_high": 160, "num_windows_y": 18, "num_windows_x": 34, "max_score": 2.9673092365264893, "mean_score": 0.08812437951564789}}}</tool_response>

<tool_call>{"name": "zoom_image", "arguments": {"image_index": 0, "x1": 520, "y1": 320, "x2": 620, "y2": 420, "scale": 3.0}}</tool_call>

<tool_response>{"status": "success", "images": "<image>", "output": {"box": [520, 320, 619, 420], "scale": 3.0}}</tool_response>



<think>The edge anomaly detection on image_index 2 (1.15s) shows a high-scoring box at [600,831,657,935] which corresponds to the logo area, similar to the previous frame but slightly shifted. More importantly, comparing the two frames, the object the person is holding has changed shape significantly between 0.85s and 1.15s. The first frame shows a flat item while the second shows a different, possibly rotated or replaced object. This rapid change in the held object across consecutive frames suggests tampering. The transition likely starts just before 0.85s and ends just after 1.15s. I should check the exact boundaries by examining the transition more closely. I'll extract a very narrow segment around 0.8s to 1.2s to get higher temporal resolution and confirm the exact start/end of the tampering.</think>

<tool_call>{"name": "extract_video_segment_frame", "arguments": {"start_sec": 0.8, "end_sec": 1.2}}</tool_call>

<tool_response>{"status": "success", "images": "<image><image>", "output": {"time_range_sec": [0.8, 1.2], "raw_frame_range": [16, 24], "raw_frame_count_in_range": 9, "sampled_frame_count_for_mllm": 2, "sampled_frame_indices_for_mllm": [16, 24], "sampled_frame_timestamps_sec_for_mllm": [0.8, 1.2], "sampled_fps_for_mllm": 4.444444, "sampled_image_count": 2}}</tool_response>



<think>The new extraction gives me frames at exactly 0.8s and 1.2s. Comparing these with the previous frames: at 0.8s the person is holding a flat item (similar to 0.85s), while at 1.2s the object has changed to the rotated/longer shape seen at 1.15s. This confirms the transition happens between 0.8s and 1.2s. The earlier edge anomalies were consistent with tampering in the object region during this interval. Since the change is abrupt and localized to the held object, this segment is likely tampered. The boundaries should be from just before the change starts (0.8s) to just after it completes (1.2s). Given the frame timestamps, the tampered segment is 0.80s-1.20s.</think>

<answer>Answer: ["0.80s-1.20s"]</answer>

Case5 - Segmentation

Question: Please locate the tampered region and use the SAM3 segmentation tool to generate the final mask.



OmniVL-Guard Pro:



<think>I notice a large region in the image that appears to contain a painting — it's visually distinct from the background and foreground elements. To assess whether this area has been tampered with, I'll start by examining its edges and internal structure for inconsistencies. I'll zoom into this region to get a closer look at potential anomalies.**</think>**



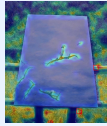
<tool_call>{"name": "zoom_image", "arguments": {"image_index": 0, "x1": 85, "y1": 143, "x2": 918, "y2": 738, "scale": 2.0}}**</tool_call>**

<tool_response>{"status": "success", "images": "<image>", "output": {"box": [85, 143, 918, 738], "scale": 2.0}}**</tool_response>**



<think>After zooming in, I observe that the painting contains two eagles perched on branches — one near the top and another near the bottom. The top eagle's head appears unusually smooth and lacks fine feather detail, which may suggest digital manipulation. To investigate further, I'll analyze the edges within this region for irregularities that could indicate tampering.**</think>**

<tool_call>{"name": "detect_edge_anomaly", "arguments": {"image_index": 0, "block_size": 32, "stride": 16, "canny_low": 50, "canny_high": 150, "top_k": 5}}**</tool_call>**



<tool_response>{"status": "success", "images": "<image>", "output": {"suspicious_boxes": [{"score": 3.427011489868164, "box": [867, 469, 914, 500], "edge_density": 0.23828125, "lap_energy": 48.861328125, "neighbor_gap": 8.59429931640625}, {"score": 3.1179940700531006, "box": [703, 750, 781], "edge_density": 0.3017578125, "lap_energy": 45.984375, "neighbor_gap": 5.92034912109375}, {"score": 3.0251986980438232, "box": [258, 875, 305, 906], "edge_density": 0.26953125, "lap_energy": 41.890625, "neighbor_gap": 6.68817138671875}, {"score": 2.9954140186309814, "box": [422, 828, 469, 859], "edge_density": 0.3369140625, "lap_energy": 54.029296875, "neighbor_gap": 3.7099609375}, {"score": 2.9882943630218506, "box": [679, 875, 726, 906], "edge_density": 0.2861328125, "lap_energy": 41.369140625, "neighbor_gap": 6.1661376953125}], "global_stats": {"block_size": 32, "stride": 16, "canny_low": 50, "canny_high": 150, "num_windows_y": 63, "num_windows_x": 41, "max_score": 3.427011489868164, "mean_score": 0.27704912424087524}}}**</tool_response>**



<think>The edge analysis reveals several suspicious areas, with the strongest signal located near the top eagle's head. This region shows subtle discontinuities in texture and edge continuity, reinforcing the suspicion of manipulation. To better understand the nature of the tampering, I'll now examine the facial structure of the top eagle more closely — if it's a real bird, we'd expect to see clear facial features; if not, it may be a synthetic insertion.**</think>**

<tool_call>{"name": "detect_faces", "arguments": {"image_index": 0, "det_size": 640, "score_threshold": 0.5, "max_faces": 5}}**</tool_call>**

<tool_response>{"status": "success", "output": {"bboxes": []}}**</tool_response>**



<think>The face detection tool returned no results, which is expected since eagles don't have human-like faces. However, the absence of detectable facial features doesn't rule out tampering — it just means the manipulation may not involve facial structure. Instead, I'll focus on the top eagle's head, where the edge anomalies were most pronounced. I'll zoom in on that specific area to inspect for signs of blending, texture mismatch, or unnatural smoothing.**</think>**



<tool_call>{"name": "zoom_image", "arguments": {"image_index": 0, "x1": 520, "y1": 320, "x2": 620, "y2": 420, "scale": 3.0}}**</tool_call>**

<tool_response>{"status": "success", "images": "<image>", "output": {"box": [520, 320, 619, 420], "scale": 3.0}}**</tool_response>**



<think>The close-up of the top eagle's head confirms my suspicion: the texture is unnaturally smooth, lacking the fine feather detail and subtle shading you'd expect in a real painting. The edges around the head also appear slightly blurred, suggesting it may have been digitally inserted or altered. Based on this, I'll define a bounding box covering the entire painting region, place a positive point on the top eagle's head — the most suspicious area — and add negative points on the background within the same box to help SAM3 distinguish the tampered region from its surroundings.**</think>**



<tool_call>{"name": "sam3_segmentation", "arguments": {"image_index": 0, "input_boxes": [[[85, 143, 918, 738]], "input_points": [[[600, 380], [100, 200], [800, 600]]], "input_labels": [[[1, 0, 0]]]}}**</tool_call>**

<tool_response>{"status": "success", "images": "<mask_image>", "output": {"mask_file": "test30k_033690_sam3_mask.png"}}**</tool_response>**

<answer>Answer: Segmentation complete. Mask generated as png.**</answer>**